IGNOU Buddy

MCS-224

Artificial Intelligence is related to the process of incorporating intelligence into machines

On the basis of the functionalities the AI can be classified based on Type 1 and Type 2

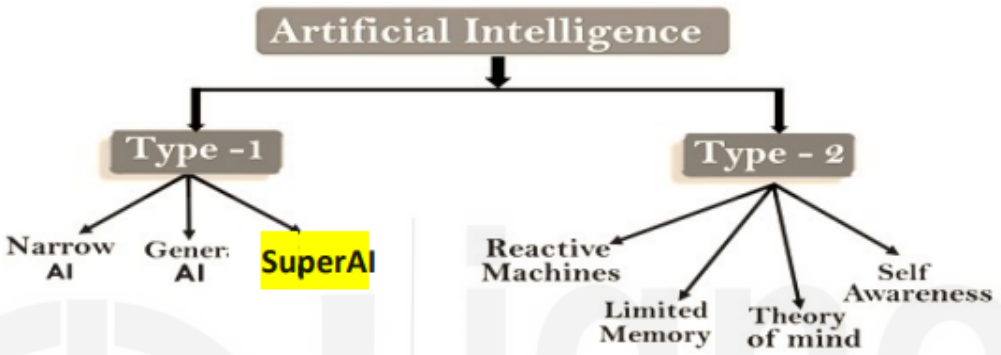
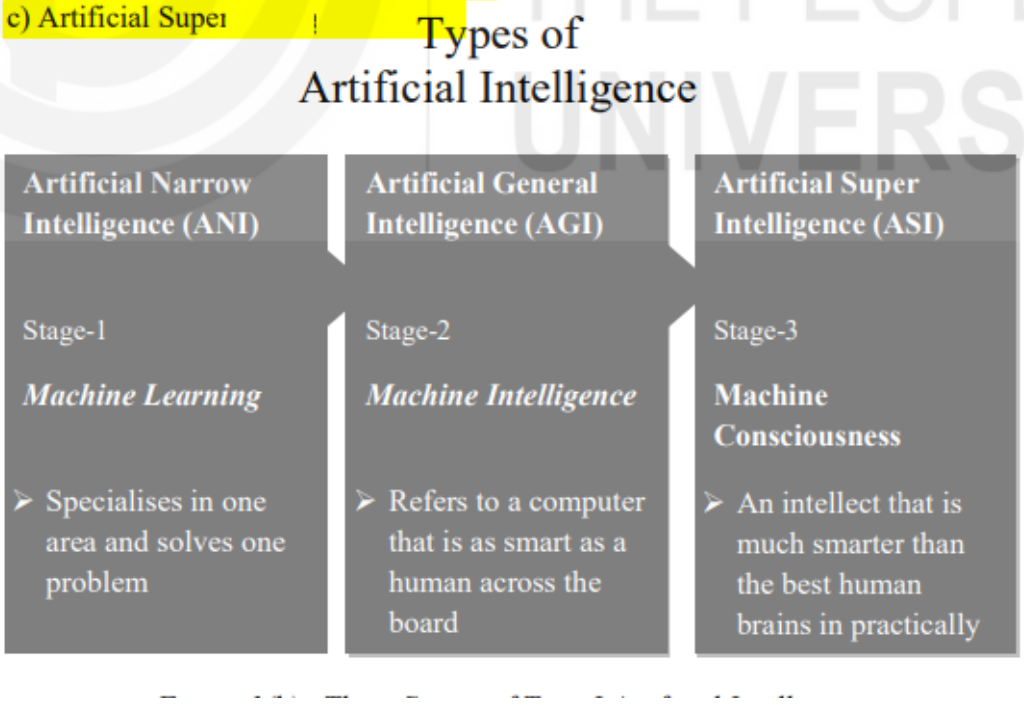


Figure 1(a) – Classification of Artificial Intelligence

Here's a brief introduction the first type of AI i.e., Type 1 AI. Following are the three stages of Type 1 - Artificial Intelligence:

a) Artificial Narrow Intelligence-(ANI)
b) Artificial General Intelligence-(AGI)
c) Artificial Super



The various categories of Artificial Intelligence are discussed as follows:

a) Artificial Narrow Intelligence (ANI), also called Weak AI or Narrow AI: Weak AI is a term for thinking that is "simulated." Such systems seem to act intelligently, but they don't have any awareness of what they are doing. For example, a chatbot might talk to you in a way that seems natural, but it doesn't know who it is or why it's talking to you. Artificial intelligence is a system that was built to do a certain job.

b) Artificial General Intelligence (AGI): Strong or General Artificial Intelligence, also called "actual" thinking. That is, acting like a smart human and thinking like one with a conscious, subjective mind. For instance, when two humans talk, they probably both know who they are, what they're doing, and why.

   Systems with strong or general artificial intelligence can do things that people can do. These systems tend to be harder to understand and more complicated. They are set up to handle situations where they might need to solve problems on their own without help from a person. Uses for these kinds of systems include self-driving cars and operating rooms in hospitals.

c) Artificial Super Intelligence (ASI) - Super intelligence: The term "super intelligence" usually refers to a level of general and strong AI that is smarter than humans, if that's even possible. The ASI is seen as the logical next step after the AGI because it can do more than humans can. This includes making decisions, making rational decisions, and even things like building emotional relationships. Their is a marginal difference between AGI and ASI.

Let's try to answer the question, "**What is the Turing Test in AI**?" Alan Turing, who is the most well-known name among the pioneers, thought about how to test an A.I. product to see if it was intelligent. Turing came up with a test, which is now known as the Turing Test, to see if something is smart. Below is a summary of the Turing test. See figure 2 for more details.

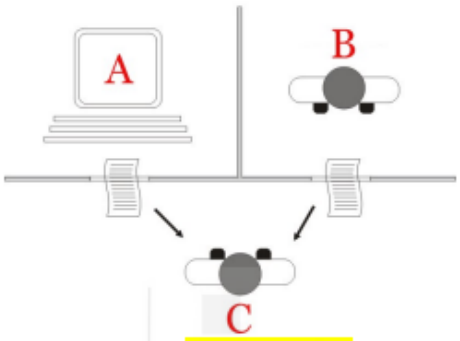


Figure 2: Turing Test

For the purpose of the test, There are three rooms that will be used for the test. In one of the rooms, there is a computer system that is said to be smart. In each of the other two rooms, there is one person sitting. One of the people, who we'll call C, is supposed to ask questions of the computer and the other person, who we'll call B, without knowing who each question is for and, of course, with the goal of figuring out who the computer is. On the other hand, the computer would reply in a way that would keep C from finding out who it is.

The only way for the three of them to talk to each other is through computer terminals. This means that the identity of the computer or person B can only be determined by how intelligent or not the responses are, not by any other human or machine traits. If C can't figure out who the computer is, then the computer must be smart. More accurately, the computer is smart if it can hide its identity from C.

Note that for a computer to be considered smart, it should be smart enough not to answer too quickly, at least not in less than a hundredth of a second, even if it can do something like find the sum of two numbers with more than 20 digits each.

Criticism to Turing Test: There have been a number of criticisms of the Turing test as a machine intelligence test. The Chinese Room Test, developed by John Searle, is one of the most well-known Criticism. The crux of the Chinese Room Test, which we'll discuss below, is that convincing a system, say A, that it possesses qualities of another system, say B, does not suggest that system A actually possesses those qualities. For example, a male human's ability to persuade people that he is a woman does not imply that he is capable of bearing children like a woman.

1.7 COMPARISON - ARTIFICIAL INTELLIGENCE, MACHINE LEARNING & DEEP LEARNING

Artificial intelligence is a big field that includes a lot of different ways of doing things, from top-down (knowledge representation) to bottom-up (machine learning). In recent years, people have often talked about three related ideas: artificial intelligence (AI), machine learning (ML), and deep learning (DL) (DL). AI is the most general term, machine learning is a part of AI, and deep learning is a type of machine learning. Figure 5 shows how these three ideas are related to each other. Figure 2(b) shows that AI is a broad field with many different subdomains. However, AI's recent rise in popularity is largely due to how well machine learning, especially deep learning, works. So, this entry will talk about these two areas of AI: Machine Learning (ML) and Deep Learning (DL) Figure 2 (a)

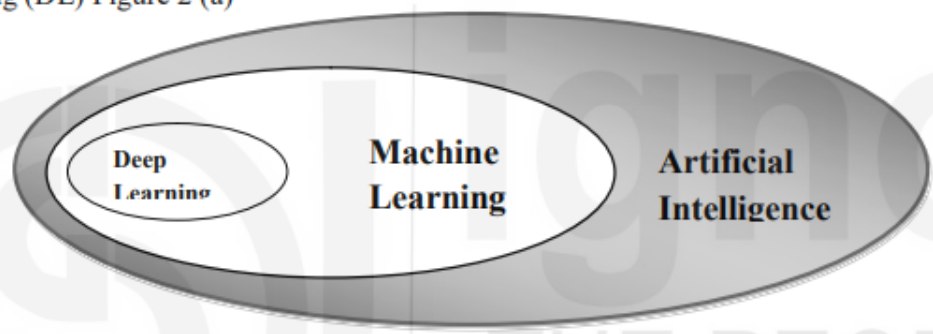


Fig 2(a):AI, ML, DL

Descriptive, Predictive, Prescriptive Analytics

ML shows a machine how to draw conclusions and make decisions based on what it has learned in the past. It looks for patterns and analyses past data to figure out what these patterns mean so that a possible conclusion can be reached without the need for human experience. Businesses save time and make better decisions by using automation to evaluate data and come to conclusions..

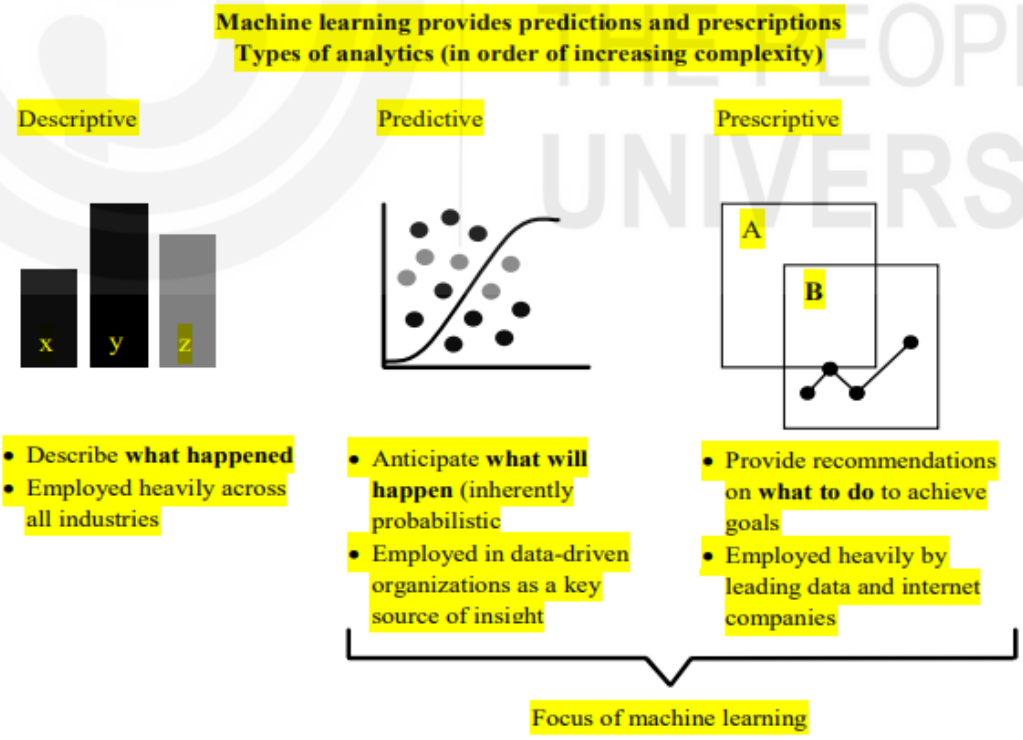


Figure 3 :Machine Learning : Descriptive, Predictive and Prescriptive Analytics

INTELLIGENT AGENTS

An agent may be thought of as an entity that acts, generally on behalf of someone else. More precisely, an agent is an entity that perceives its environment through sensors and acts on the environment through actuators.

According to the second approach to defining agent, an agent is supposed to possess some or all of the following properties:

- Reactivity: The ability of sensing the environment and then acting accordingly.
- Autonomy: The ability of moving towards its goal, changing its moves or strategy, if required, without much human intervention.
- Communicating ability: The ability to communicate with other agents and humans.
- Ability to coexist by cooperating: The ability to work in a multi-agent environment to achieve a common goal.
- Ability to adapt to a new situation: Ability to learn, change and adapt to the situations in the world around it.
- Ability to draw inferences: The ability to infer or conclude facts, which may be useful, but are not available directly.
- Temporal continuity: The ability to work over long periods of time.
- Personality: Ability to impersonate or simulate someone, on whose behalf the agent is acting.
- Mobility: Ability to move from one environment to another.

In context of Intelligent Agents, what are task environments ? Explain the standard set of measures for specifying a task environment under the heading PEAS.

Task environments or problem environments are the environments, which include all the elements involved in the problems for which agents are thought of as solutions. Task environments will vary with every new task or problem for which an agent is being designed. Specifying the task environment is a long process which involves looking at different measures or parameters. Next, we discuss a standard set of measures or parameters for specifying a task environment under the heading PEAS.

PEAS (Performance, Environment, Actuators, Sensors)

For designing an agent, the first requirement is to specify the task environment to the maximum extent possible. The task environment for an agent to solve one type of problems, may be described by the four major parameters namely, performance (which is actually the expected performance), environment (i.e., the world around the agent), actuators (which include entities through which the agent may perform actions) and sensors (which describes the different entities through which the agent will gather information about the environment).

The four parameters may be collectively called as PEAS. We explain these parameters further, through an example of an automated agent, which we will preferably call automated public road transport driver. This is a much more complex agent than the simple boundary following robot which we have already discussed.

Next, we discuss some of the general categories of agents based on their agents' programs, Agents can be grouped into five classes based on their degree of perceived intelligence and capability. All these agents can improve their performance and generate better action over the time. These are given below:

- ***SR (Simple Reflex) agents***
- ***Model Based reflex agents***
- ***Goal-based agents***
- ***Utility based agents***
- ***Stimulus-Response Agents***
- ***Learning agents***

SR (Simple Reflex) agents: These are the agents or machines that have no internal state (i.e., the don't remember anything) and simply react to the current percepts in their environments. An interesting set of agents can be built, the behaviour of the agents in which can be captured in the form of a simple set of functions of their sensory inputs. One of the earliest implemented agents of this category was called ***Machina Speculatrix .*** This was a device with wheels, motor, photo cells and vacuum tubes and was designed to move in the direction of light of less intensity and was designed to avoid the direction of the bright light. **A boundary following robot is also an SR agent**. For an automobile-driving agent also, some aspects of its behavior like applying brakes immediately on observing either the vehicle immediately ahead applying brakes or a human being coming just in front of the automobile suddenly,

show the simple reflex capability of the agent. Such a simple reflex action in the agent program of the agent can be implemented with the help of simple condition-action rules.

For example : ***IF*** *a human being comes in front of the automobile suddenly*
THEN *apply breaks immediately*.

Although implementation of SR agents is simple **yet on the negative side** this type of agents has very limited intelligence because ***they do not store or remember anything.*** As a consequence, they cannot make use of any previous experience. In summary, they do not learn. Also **they are capable of operating correctly only if the environment is fully observable.**

Model Based Reflex agents : Simple Reflex agents are not capable of handling task environments that are not fully observable. In order to handle such environments properly, in addition to reflex capabilities, the agent should, maintain some sort of internal state in the form of a function of the sequence of percepts recovered up to the time of action by the agent. Using the percept sequence, the internal state is determined in such a manner that it reflects some of the aspects of the unobservable environment. Further, in order to reflect properly the unobserved environment, the agent is expected to have a model of the task environment encoded in the agent's program, where the model has the knowledge about–

(i) the process by which the task environment evolves independent of the agent and
(ii) effects of the actions of the agent have on the environment.
Thus, in order to handle properly the partial observability of the environment, the agent should have a model of the task environment in addition to reflex capabilities. Such agents are called **Model-based Reflex Agents**

2.2 Introduction to State Space Search

It is necessary to represent an AI problem in the form of a state space before it can be solved. A state space is the set of all states reachable from the initial state. A state space forms a graph in which the nodes are states and the arcs between nodes are actions. In state space, a path is a sequence of states connected by a sequence of actions.

2.2.3 Problem formulation:

Many problems can be represented as ***state space***. The state space of a problem includes: an *initial state*, one or more *goal state*, set of *state transition operator* (or a ***set of production rules***), used to change the current state to another state. This is also known as ***actions.*** A ***control strategy*** is used that specifies the order in which the rules will be applied. For example, Depth-first search (DFS), Breath-first search (BFS) etc. It helps to find the goal state or a path to the goal state.

In general, a state space is represented by 4 tuples as follows:$\boldsymbol{S_s}$: $[\boldsymbol{S}, \boldsymbol{s_0}, \boldsymbol{O}, \boldsymbol{G}]$

Where **S**: Set of all possible states.
$\boldsymbol{s_0}$**:** start state (initial configuration) of the problem, $s_0 \in S$.
O: Set of production rules (or set of state transition operator) used to change the state from one state to another. It is the set of arcs (or links) between nodes.

The ***production rule*** is represented in the form of a pair. Each pair consists of a left side that determines the applicability of the rule and a right side that describes the action to be performed, if the rule is applied.

G: Set of Goal state, $G \in S$.

The sequence of actions (or operators) is called a solution path. It is a path from the initial state to a goal state. This sequence of actions leads to a number of states, starting from initial state to a goal state, as $\{s_0, s_1, s_2, \dots\dots, s_n \in G\}$.A sequence of state is called a path. The cost of a path is a positive number. In most of the cases the path cost is computed as the sum of the costs of each action.
The following figure 3 shows a search process in a given state space.

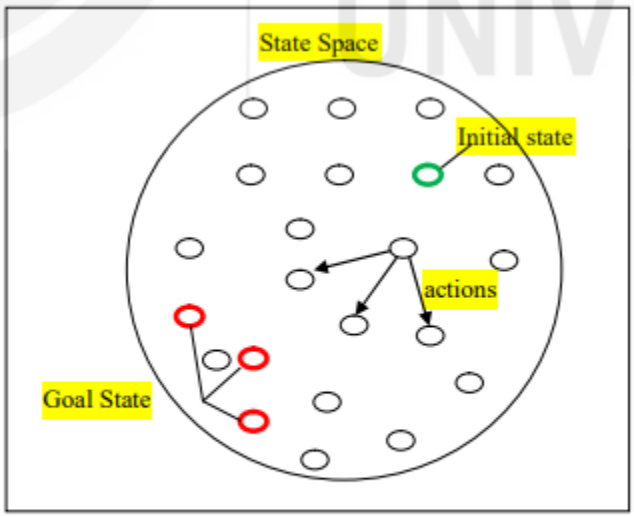


Fig 3 State space with initial and goal node

Water Jug Problem:

Example1: State space representation of Water-Jug problem (WJP):

Problem statement:
Given two jugs, a 4-gallon and a 3-gallon, both of which do not have measuring indicators on them. The jugs can be filled with water with the help of a pump that is available (as shown in figure 4)

The question is "how can you get exactly 2 gallons of water into 4-gallon jug".

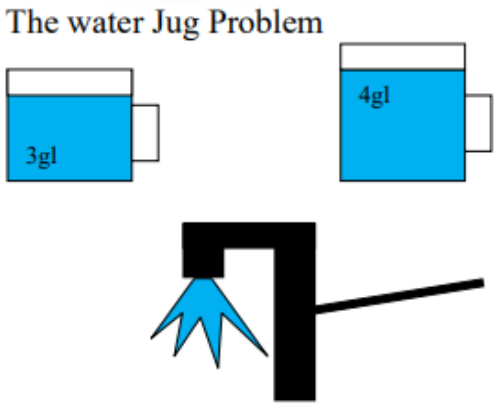


Fig 4 The Water Jug problem

The state space or production rule of this problem can be defined as a collection of ordered pairs of integers (x,y) where x=0,1,2,3,4and y= 0,1,2,or 3.
In the order pair (x,y), x is the amount of water in four-gallon jug and y is the amount of water in the three-gallon jug.
State space: all possible combination of (x,y)
The **start state** is (0,0) and
Goalstate is $(2, n), where\ n = 0,1,2,3$

Solution-1:
Table-2Getting exactly 2 gallons of water into 4-gallon jug (solution1)

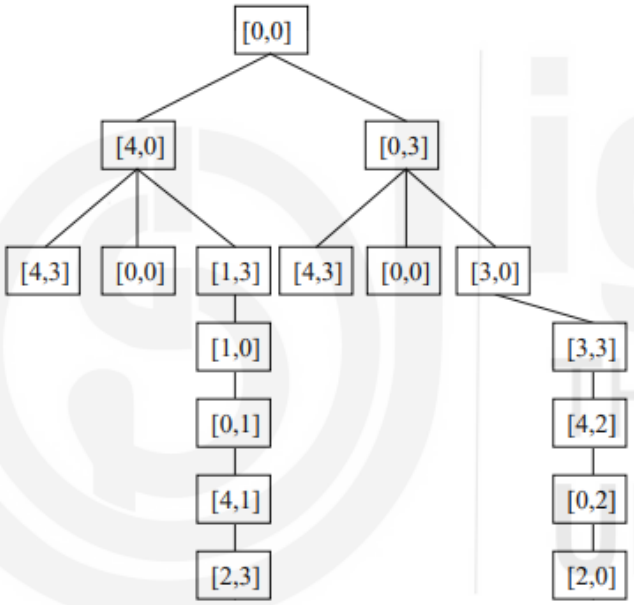
4-gal jug	**3-gal jug**	**Rule applied**
0	0	Initial state
4	0	R1 {fill 4-gal jug}
1	3	R6 {Pure all water from 4-gal jug to 3-gal jug
1	0	R4 {empty 3-gal jug}
0	1	R8 {Pour water from 4 to 3-gal jug}
4	1	R1 {Fill 4-gal jug}
2	3	R6 {Pour water from 4-gal jug to 3-gal jug until it is full}
2	0	R4 {empty 3-gal jug}

Solution-2

Table-3Getting exactly 2 gallons of water into 4-gallon jug (solution2)

4-gal jug	3-gal jug	Rule applied
0	0	Initial state
0	3	R2 {fill 3-gal jug}
3	0	R7 {Pure all water from 3-gal jug to 4-gal jug
3	3	R2 {fill 3-gal jug}
4	2	R5 {Pour from 3 to 4-gal jug until it is full}
0	2	R3 {Empty 4-gal jug}
2	0	R7 {Pour all water from 3-gal jug to 4-gal jug}

A state space tree for WJP with all possible solution is shown in figure 7.



*** Briefly discuss the Adversarial search. Name the techniques used for adversarial search.**

2.5 Adversarial Search-Two agent search

In computer science, a search algorithm is an algorithm for finding an item with specified properties among a collection of items. The items may be stored individually as records in a database; or may be elements of a search space defined by a mathematical formula or procedure.
Adversarial search is a **game-playing** technique where the agents are surrounded by a competitive environment. A conflicting goal is given to the agents (multiagent). These agents compete with one another and try to defeat one another in order to win the game. Such conflicting goals give rise to the adversarial search. Here, game-playing means discussing those games where **human intelligence** and **logic factor** is used, excluding other factors such as **luck factor**. **Tic-tac-toe, chess, checkers**, etc., are such type of games where no luck factor works, only mind works.
Mathematically, this search is based on the concept of **'Game Theory.'** ***According to game theory, a game is played between two players. To complete the game, one has to win the game and the other loses automatically.'***

We are opponents- I win, you loose.

Techniques required to get the best optimal solution
There is always a need to choose those algorithms which provide the best optimal solution in a limited time. So, we use the following techniques which could fulfil our requirements:

- **Pruning:** A technique which allows ignoring the unwanted portions of a search tree which make no difference in its final result.
- **Heuristic Evaluation Function:** It allows to approximate the cost value at each level of the search tree, before reaching the goal node.

2.5.2 Issues in Adversarial search

In a **normal search**, we follow a sequence of actions to reach the goal or to finish the game optimally. But in an **adversarial search**, the result depends on the players which will decide the result of the game. It is also obvious that the solution for the goal state will be an optimal solution because the player will try to win the game with the shortest path and under limited time. **Minimaxsearch Algorithm**is an example of adversarial search. **Alpha-beta Pruning** in this is used to reduce search space.

What is Min-Max Search Strategy ? Write MINIMAX algorithm.

2.6 Min-Max search strategy

In artificial intelligence, minimax is a **decision-making** strategy under **game theory,** which is used to minimize the losing chances in a game and to maximize the winning chances. This strategy is also known as '**Min-Max,' 'MM,' or 'Saddle point.'** Basically, it is a two-player game strategy where *if one wins, the other lose the game*. This strategy simulates those games that we play in our day-to-day life. Like, if two persons are playing chess, the result will be in favour of one player and will againstthe other one. The person who will make his best *try, efforts as well as cleverness, will surely win.*

We can easily understand this strategy via **game tree**-where nodes are the states of the game, and edges are moves that were made *by the players in the game*. Players will be two namely:

- **MIN:** Decrease the chances of **MAX** to win the game.
- **MAX:** Increases his chances of winning the game.

They both play the game alternatively, i.e., turn by turn and following the above strategy, i.e., if one wins, the other will definitely lose it. Both players look at one another as competitors and will try to defeat one-another, giving their best.

In minimax strategy, the result of the game or the utility value is generated by a **heuristic function** by propagating from the initial node to the root node. It follows the **backtracking technique** and backtracks to find the best choice. MAX will choose that path which will increase its utility value and MIN will choose the opposite path which could help it to minimize MAX's utility value.

2.6.1 MINIMAX Algorithm:

MINIMAX algorithm is a backtracking algorithm where it backtracks to pick the best move out of several choices. MINIMAX strategy follows the **DFS (Depth-first search)** concept. Here, we have two players **MIN and MAX,** and the game is played alternatively between them, i.e., when **MAX** made a move, then the next turn is of **MIN.** It means the move made by MAX is fixed and, he cannot change it. The same concept is followed in DFS strategy, i.e., we follow the same path and cannot change in the middle. That's why in MINIMAX algorithm, instead of BFS, we follow DFS. The following steps are used in MINMAX algorithm:

- Generate the whole game tree, all the way down to the terminal states.

- Apply the utility function to each terminal state to get its value.
- Use the utility of the terminal states to determine the utility of the nodes one level higher up in the search tree.
- Continue backing up the values from the leaf nodes toward the root, one layer at a time.
- Eventually, the backed-up values reached the top of the tree; at that point, MAX chooses the move that leads to the highest value.

This is called a minimax decision, because it maximizes the utility under the assumption that the opponent will play perfectly to minimize it. To better understand the concept, consider the following game tree or search tree as shown in figure 18.

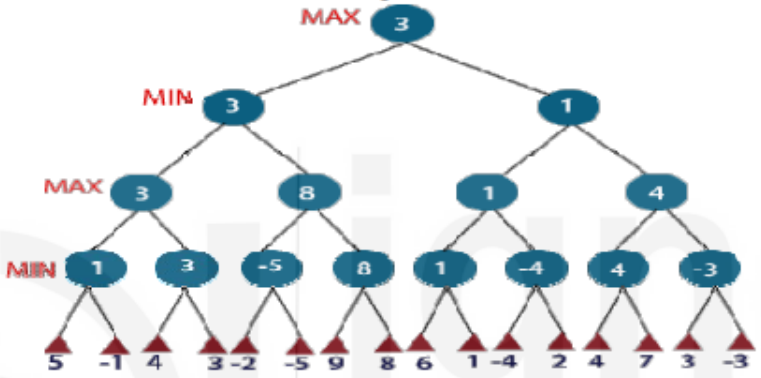


Fig 18 Two player game tree

In the above figure 2, the two players **MAX** and **MIN** are there. **MAX** starts the game by choosing one path and propagating all the nodes of that path. Now, **MAX** will backtrack to the initial node and choose the best path where his utility value will be the maximum. After this, its **MIN** chance. **MIN** will also propagate through a path and again will backtrack, but **MIN** will choose the path which could minimize **MAX** winning chances or the utility value.

So, if the level is minimizing, the node will accept the minimum value from the successor nodes. If the level is maximizing, the node will accept the maximum value from the successor.

2.6.3 Properties of Minimax Algorithm:

1. **Complete:** Minimax algorithm is complete, if the tree is finite.
2. Optimal: If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution is said to be an optimal solution. Minimax is Optimal.
3. **Time complexity:** $O(b^m)$, where b: Branching Factor and m is the maximum depth of the game tree.
4. **Space complexity:** $O(bm)$

For example, in chess playing game b = 35, m ≈100 for "reasonable" games. In this case exact solution completely infeasible

2.6.4 Advantages and disadvantages of Minimax search

Advantages:

- Returns an optimal action, assuming perfect opponent play.
- Minimax is the simplest possible (reasonable) game search algorithm.

Disadvantages:

- It's completely infeasible in practice.
- When the search tree is too large, we need to limit the search depth and apply an evaluation function to the cut-off states.

Informed search is also called as Heuristic (or guided) search. These are the search techniques where additional information about the problem is provided in order to guide the search in a specific direction. A heuristic is a method that might not always find the best solution but is guaranteed to find a good solution in reasonable time. By sacrificing completeness, it increases efficiency.

The following table summarizes the differences between uninformed and informed search:

Uninformed Search	**Informed Search**
No information about the path, cost, from the current state to the goal state. It doesn't use domain specific knowledge for searching process.	The path cost from current state to goal state is calculated, to select the minimum cost path as the next state. It uses domain specific knowledge for the searching process.
It finds solution slow as compared to informed	It finds solution more quickly.
Less efficient	More efficient
Cost is high.	Cost is low
No suggestion is given regarding the solution init. Problem to be solved with the given information only.	It provides the direction regarding the solution. Additional information can be added as assumption to solve the problem.
Examples are: ▪ Depth First Search, ▪ Breadth First Search, ▪ Depth limited search, ▪ Iterative Deepening DFS, ▪ Bi-directional search	Examples are: ▪ Best first search ▪ Greedy search ▪ A* search

3.3.1 Breadth-first search (BFS):

It is the simplest form of blind search. In this technique the root node is expanded first, then all its successors are expanded and then their successors and so on. **In general, in BFS, all nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded.** It means that all immediate children of nodes are explored before any of the children's children are considered. The search tree generated by BFS is shown below in Fig 2.

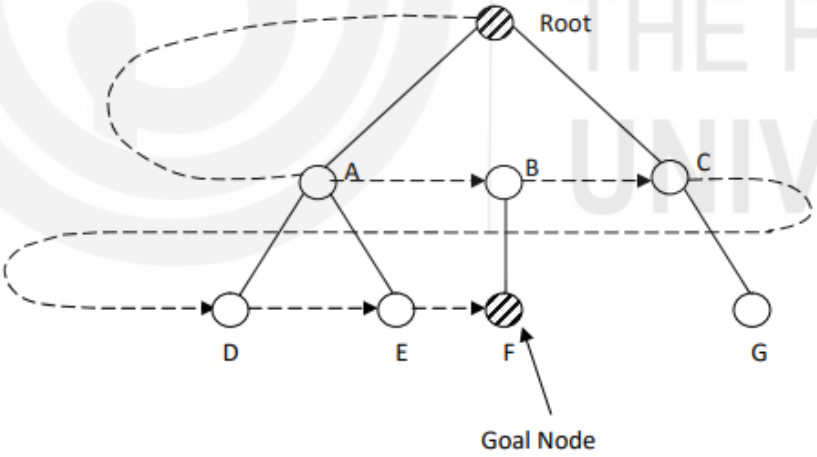


Fig 2 Search tree for BFS

Note that BFS is a brute-search, so it generates all the nodes tor identifying the goal and note that we are using the convention that the alternatives are tried in the left-to-right order.

A BFS algorithm uses a data structure-**queue** that works on FIFO principle. This queue will hold all generated but still unexplored nodes. **Please remember that the order in which nodes are placed on the queue or removal and exploration determines the type of search.**

We can implement it by using two lists called OPEN and CLOSED. The OPEN list contains those states that are to be expanded and CLOSED list keeps track of state already expanded. Here OPEN list is used as a **queue.**

BFS is effective when the search tree has a low branching factor.

Breath-First Search (BFS) Algorithm
1. Initialize: Set = $\{s\}$, where s is a start state.
2.Fail: If $OPEN = \{\ \}$, terminate with failure.
3. Select: Remove a left most state (say $\boldsymbol{a}$) from OPEN.
4.Terminate: If $a \in Goal\ node$, terminate with success, else
5. Expend: Generate the successor of node $\boldsymbol{a}$, discard the successors of $\boldsymbol{a}$ if it's already in OPEN, Insert only remaining successors on right end of OPEN [i.e., QUEUE]
6. LOOP: Goto Step 2

Performance of BFS:

- **Time** Required for BFS for tree of b branching factor and d depth is $O(b^d)$.
- **Space** (memory) requirement for a tree with b branching factor and d depth is also $O(b^d)$
- BFS algorithm is **Complete** (if b is finite).
- BFS algorithm is **Optimal** (if cost = 1 per step)

Space is the bigger problem in BFS as compared to DFS.

3.3.3 Advantages and disadvantages of BFS:

Advantages: BFS has some advantages and are given below

1. BFS will never get trapped exploring bund alley.

2. It is guaranteed to find a solution if one exists.

Disadvantages: BFS has certain disadvantages also. They are given below-

1. Time complexity and Space complexity are both $O(b^d)$ i.e., exponential type. This is very hurdle.

2. All nodes are to be generated in BFS. So, even unwanted nodes are to be remembered (stored in queue) which is of no practical use of the search.

3.3.4 Depth First Search

A Depth-First Search (DFS) explores a path all the way to a leaf before backtracking and exploring another path. That is expand deepest unexpanded node (expand most recently generated deepest node first).

The search tree generated by the DFS is show in figure below:

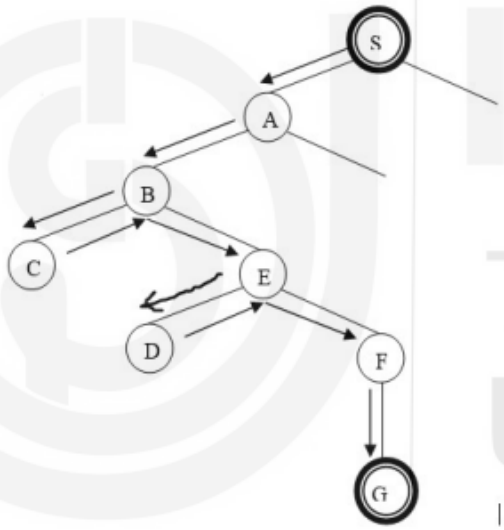


Fig 4 Depth first search (DFS) tree

In depth-first search we go as far down as possible into the search tree/graph before backing up and trying alternatives. It works by always generating a descendent of the most recently expanded node until some depth cut off is reached and then backtracks to next most recently expanded node and generates one of its descendants. So only path of nodes from the initial node to the current node is stored, in order to execute the algorithm. For example, consider the following tree and see how the nodes are expended using DFS algorithm.

Depth-First Search (DFS) Algorithm

1. Initialize: Set $= \{s\}$, where s is a start state.

2.Fail: If $OPEN = \{\ \}$, terminate with failure.

3. Select: Remove a left most state (say $\boldsymbol{a}$) from OPEN.

4.Terminate: If $a \in Goal\ node$, terminate with success, else

5. Expend: Generate the successor of node $\boldsymbol{a}$, discard the successors of $\boldsymbol{a}$ if it's already in OPEN, Insert remaining successors on **left** end of OPEN [i.e., STACK]

6. LOOP: Goto Step 2

Note: The only difference between BFS and DFS is in **Expend** (step 5). In BFS, we always insert the generated successors at the right end of OPEN list, whereas in DFS at the left end of OPEN list.

3.3.6 Advantages and disadvantages of DFS:

Advantages:

- If depth-first search finds solution without exploring much in a path then the time and space it takes will be very less.
- The advantage of depth-first Search is that memory requirement is only linear with respect to the search graph. This is in contrast with breadth-first search which requires more space.

3.4 Iterative Deepening Depth First Search (IDDFS)

Iterative Deepening Depth First Search (IDDFS) neither suffers the drawbacks of BFS nor DFS on trees. It takes the advantages of both the strategy.
It begins by performing DFS to a depth of zero, then depth of one, depth of two, and so on until a solution is found or some maximum depth is reached.
It is like BFS in that it explores a complete layer of new nodes at each iteration before going to next layer.
It is likes DFS for a single iteration.

It is preferred when there is a large search space, and the depth of a solution is notknown. But it performs the wasted computation before reaching the goal depth. Since IDDFS expends all nodes at a given depth before expending any nides at greater depth, it is guaranteed to find a shortest-length (path) solution from initial state to goal state.
At any given time, it is performing a DFS and never searches deeper than depth 'd'. Hence, it uses same space as DFS.
Disadvantage of IDDFS is that it performs wasted computation prior to reaching the goal depth.

Algorithm (IDDFS)

Initialized $d = 1/*$ depth of search tree $*/$, found $=$ false
While $(Found = False)$
DO{
    perform a depth first search from start to depth d.
    if goal state is obtained
    then Found = true
else

discard the nodes generated in the search after depth d
(i.e. d + 1 onwards till last of tree)
}$/*$ *end while* $*/$

Report the solution, if obtained
Stop

3.4.1 Time and space complexities of IDDFS

The time and space complexities of IDDFSalgorithm is $O(b^d)$ and O(d) respectively.

It can be shown that depth first iterative deepening is **asymptotically optimal,** among brute force tree searches, in terms of time, space and length of the solution. In fact, it is linear in its space complexity like DFS, and is asymptotically optimal to BFS in terms of the number of nodes expanded.

Please note that in general iterative deepening is preferred uninformed search method when there is large search space, and the depth of the solution is unknown. Also note that iterative deepening search is analogous to BFS in that is explores a complete layer going to the next layer.

3.4.2 Advantages and Disadvantages of IDDFS:

Advantages:

1. It combines the benefits of BFS and DFS search algorithms in terms of fast search and memory efficiency.
2. It is guaranteed to find a shortest path solution.
3. It is a preferred uniformed search method when the search space is large and the depth of the solution is not known.

Disadvantages

1. The main drawback of IDDFS is that it repeats all the work from the previous phase. That is, it performs wasted computations before reaching the goal depth.
2. The time complexity is $O(b^d)$ i.e., exponential type only.

The following summarizes when to use which algorithm:

DFS	BFS	IDDFS
Many solutions exist Know (or have a good estimate of) the depth of solution.	Some solutions are known to be shallow	Space is limited and the shortest solution path is required

3.7.3 Best-First Search

Best first search uses an evaluation function f(n) that gives an indication of which node to expand next for each node. Every node in a search space has an evaluation function (heuristic function) associated with it. A heuristic function value h(n) on each node indicates how the node is from the goal node. Note that Evaluation function=heuristic cost function (in case of minimization problem) OR objective function(in case of maximization).Decision of which node to be expanded depends onvalue of evaluation function. Evaluation value= cost/distance of current node from goal node and for goal node evaluation function value=0

Based on the evaluation function, f(n), Best-first search can be categorized into the following categories:

1) Greedy Best first search
2) A* search

The following 2 list (**OPEN** and **CLOSED**) are maintained to implement these two algorithms.

1. **OPEN** – all those nodes that have been generated & have has heuristic function applied to them but have not yet been examined.
2. **CLOSED**- contains all nodes that have already been examined.

3.7.4Greedy Best-First search:

Greedy best-first search algorithm always selects the path which appears best at that moment. It is the combination of depth-first search and breadth-first search algorithms. It uses the heuristic function and search. Best-first search allows us to take the advantages of both algorithms. With the help of best-first search, at each step, we can choose the most promising node. In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e.

f(n)= g(n).

Were, h(n)= estimated cost from node n to the goal.

The greedy best first algorithm is implemented by the **priority queue** (or to store the heuristic function value).

The greedy best first algorithm is implemented by the **priority queue** (or to store the heuristic function value).

Best first search algorithm:

1. Initialize: Set $OPEN = \{s\}$, CLOSED={ };

$$f(s) = h(s)$$

2.Fail: If $OPEN = \{\}$, terminate with failure.

3. Select: Select the minimum cast state $\boldsymbol{a}$ from OPEN, save n in CLOSED.

4.Terminate: If $a \in Goal\ node$, terminate with success and return f(n), else

5. Expend: For each successor, m of n,

If $m \notin [OPEN \cup CLOSED]$

$Set\ f(m) = h(m)\ and\ \ Insert\ m\ in\ OPEN$

.....

6. LOOP: Goto Step 2

OR Pseudocode (for Best-First Search algorithm)

Step 1: Place the starting node into the OPEN list.
Step 2: If the OPEN list is empty, Stop and return failure.
Step 3: Remove the node n, from the OPEN list which has the lowest value of h(n), and places it in the CLOSED list.
Step 4: Expand the node n, and generate the successors of node n.
Step 5: Check each successor of node n, and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.

Step 6: For each successor node, algorithm checks for evaluation function f(n), and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both lists, then add it to the OPEN list.
Step 7: Return to Step 2.

Advantages of Best-First search:

- Best first search can switch between BFS and DFS, thus gaining the advantages of both the algorithms.
- This algorithm is more efficient than BFS and DFS algorithms.

Disadvantages of Best-First search:

- Chances of getting stuck in a loop are higher.
- It can behave as an unguided depth-first search in the worst-case scenario.

Evaluation of Best-First Search algorithm:

Time Complexity:

The worst-case time complexity of Greedy best first search is $O(b^m)$.

Space Complexity:

The worst-case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.

Complete: Greedy best-first search is also incomplete, even if the given state space is finite.

Optimal: Greedy best first search algorithm is not optimal.

3.8 A* Algorithm

A* search is the most commonly known form of best-first search. It uses heuristic function h(n), and cost to reach the node n from the start state g(n). It has combined features of uniform cost search (UCS) and greedy best-first search, by which it solves the problem efficiently. A* search algorithm finds the shortest path through the search space using the heuristic function. This search algorithm expands less search tree and provides optimal result faster.

In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence, we can combine both costs as following, and this sum is called as a **fitness number (Evaluation Function)**.

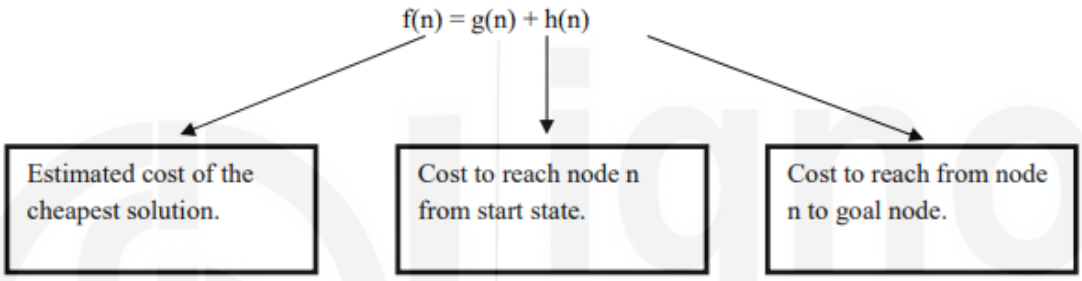


Fig 9 Evaluation function f(n) in A* Algorithm

A* algorithm **evaluation function** $\boldsymbol{f(n)}$is defined as $\boldsymbol{f(n) = g(n) + h(n)}$

Where $\boldsymbol{g(n)}$ =sum of edge costs from start state to n

And $\boldsymbol{h(n)}$ = estimate of lowest cost path from node n→goal node.

If h(n) is **admissible** then search will find optimal solution. Admissible means underestimates cost of any solution which can reached from node. In other words, a heuristic is called admissible if it always ***underestimates***, that is, we always have $h(n) \leq h^*(n)$, where h*(n) denotes the minimum distance to a goal state from state n.

A* search begins at root node and then search continues by visiting the next node which has the least evaluation value f(n).

It evaluates nodes by using the following evaluation function

f(n) = h(n) +g(n) = estimated cost of the cheapest solution through n.

Where,

g(n): the actual shortest distance traveled from initial node to current node, it helps to avoid expanding paths that are already expansive

h(n): the estimated (or "heuristic") distance from current node to goal, it estimates which node is closest to the goal node.

Nodes are visited in this manner until a goal is reached.

Suppose s is a start state then calculation of evaluation function f(n) for any node n is shown in following figure 10.

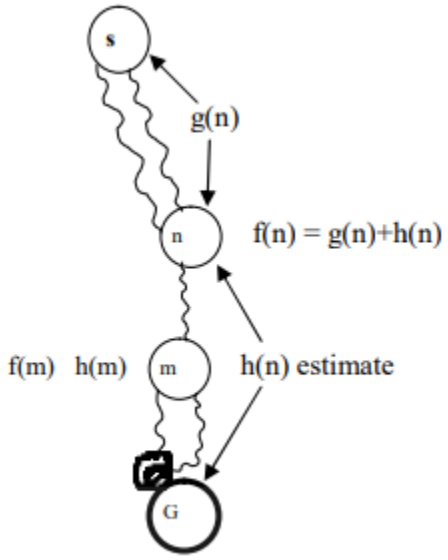


Fig 10 Calculation of evaluation function f(n).

Algorithm A*

1. **Initialize:** Set OPEN = (s), CLOSED = {}, g(s) = 0, f(s) = h(s)
2. **Fail:** If OPEN = { }, Terminate & fail
3. **Select :** Select the minimum cost state, n, from OPEN, Save n in CLOSED.
4. **Terminate**: If n ∈ G, terminate with success, and return f(n)
5. **Expand:** For each successor, m, of n

   If m ∉ [OPEN ∪ CLOSED]
   Set g(m) = g(n) + C(n,m)
   Set f(m) = g(m) +h(m)
   Insert m in OPEN

   If m ∈ [OPEN ∪ CLOSED]
   Set g(m) = min {min {g(m), g(n) + C(n,m)}
   Set f(m) = g(m) + h(m)

   If f(m) has decreased and m ∈ CLOSED,
   move m to OPEN

3.8.2 Advantages and disadvantages of A* algorithm

Advantages:

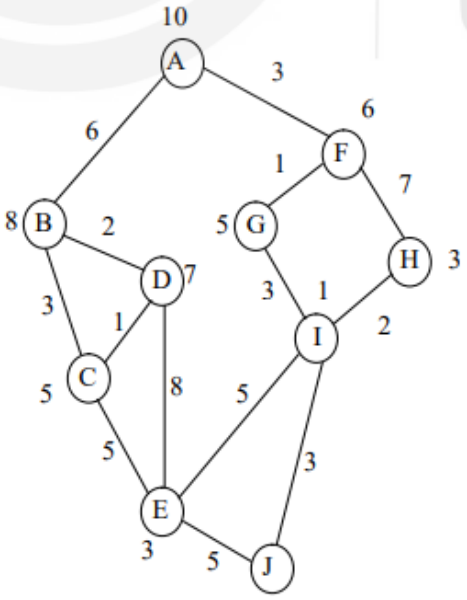
- A* search algorithm is the best algorithm than other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

Disadvantages:

- It does not always produce the shortest path as it mostly based on heuristics and approximation.
- A* search algorithm has some complexity issues.
- The main drawback of A* is memory requirement as it keeps all generated nodes i memory, so it is not practical for various large-scale problems.

3.8.1 Working of A* algorithm

Example1: Let's us consider the following graph to understand the working of A* algorithm. The numbers written on edges represent the distance between the nodes. The numbers written on nodes represent the heuristic value. Find the most cost-effective path to reach from start state **A** to **final** state **J** using A* Algorithm.



Step-1:
We start with node A. Node B and Node F can be reached from node A. A* Algorithm calculates f(B) and f(F). Estimated Cost f(n) = g(n) +h(n) for Node B and Node F is:
f(B) = 6+8=14
f(F) = 3+6=9

Since f(F) < f(F), so it decides to go to node F.
➔ Closed list (F)
Path- A →F
Step-2: Node G and Node H can be reached from node F.
A* Algorithm calculates f(G) and f(H).
f(G) = (3+1)=5=9
f(H) = (3+7) +5=13
Since f(G) < f(H), so it decides to go to node G.
➔ Closed list (G)
Path- A→F→G
Step-3: Node I can be reached from node G.
A* Algorithm calculates f(I).
f(I)=(3+1+3)+1=8; It decides to go to node I.
➔ Closed list (I).
Path- A→F→G→I

Step-4: Node E, Node H and Node J can be reached from node I.
A* Algorithm calculates f(E), f(H), f(J).
f(E) = (3+1+3+5) + 3 =15
f(H) = (3+1+3+2) + 3 = 12
f(J) = (3+1+3+3) +0 = 10
Since f(J) is least, so it decides to go to node J.
➔ Closed list (J)
Shortest Path - A→F→G→I→J
Path Cost is 3+1+3+3=10

3.9.2 AO* algorithm

Our real-life situations cannot be exactly decomposed into either AND tree or OR tree but is always combination of both. So, we need an AO* algorithm where O stands for 'ordered'. Instead of two lists OPEN and CLOSED of A* algorithm, we use a single structure GRAPH in AO* algorithm. If represents a part of the search graph that has been explicitly generated so far. Please note that each node in the graph will point both down to its immediate successors and to its immediate predecessors. Also note that each node will have some h'(n) value associated with it. But unlikeA* search, g(n) is not stored. It is not possible to compute a single value of g(n) due to many paths to the same state. It is not required also as we are doing top-down traversing along best-knownpath.

This guarantees that only those nodes that are on the best path are considered for expansion

Hence, h'(n) will only serve as the estimate of goodness of a node.

Next, we develop an AO* algorithm.

Algorithm AO*

1. Initialize: Set G* ={s}, f(s) =h(s)
   If s ∈ T, label s as SOLVED
2. Terminate: If s is SOLVED, then Terminate
3. Select: Select a non-terminal leaf node n from the marked sub-free
4. Expand: Make explicit the successors of n for each new successors, m:
   Set f(m) =h(m)
   If m is terminal, label m SOLVED
5. Cost Revision: Call **Cost-Revise (n)**
6. Loop: Go to Step 2.

Cost Revision in AO*: Cost-Revise(n)

1. Create Z = {n}
2. If Z = {} return
3. Select a node m from Z such that m has no descendants in Z
4. If m is an AND node with successors $r1, r2, \ldots rk$:

Set $f(m) = \sum [f(r_i) + c(m, r_i)]$
Mark the edge to each successor of m
If each successor is labelled SOLVED,
then label m as SOLVED.

5. If m is an OR node with successor $r1, r2, \ldots rk$:

Set $f(m) = min\{f(r_i) + c(m, r_i)\}$
Mark the edge to the best successor of m
If the marked successor is labelled SOLVED, label m as SOLVED

6. If the cost or label of m has changes, then insert those parents of m into Z for which ma is a marked successor

3.9.3 Advantage of AO* algorithm

Note that **AO* will always find a minimum cost solution if one exists if** $h'(n) < h(n)$ and that all arc costs are positive. The efficiency of this algorithm will depend on how closely $h'(n)$ approximates $h(n)$. Also note that **AO* is guaranteed to terminate** even on graphs that have cycles.

Note: When the graph has only OR node then AO* algorithm works just like A* algorithm.

4.7.1 Modus Ponens (MP)

It states that if the propositions A → B and A are true, then B is also true. Modus Ponens is also referred to as the implication elimination because it eliminates the implication A → B and results in only the proposition B. It also affirms the truthfulness of antecedent. It is written as:

α_1 → α_2, α_1 =>α_2

4.7.2 Modus Tollens (MT)

It states that if A → B and ¬ B are true then ¬ A is also true. Modus Tollens is also referred to as denying the consequent as it denies the truthfulness of the consequent. The rule is expressed as:

α_1 → α_2, ~α_2 =>~α_1

Let's begin with the Rule Based Systems:

Rather than representing knowledge in a declarative and somewhat static way (as a set of statements, each of which is true), rule-based systems represent knowledge in terms of a set of rules each of which specifies the conclusion that could be reached or derived under given conditions or in different situations.
A rule-based system consists of

(i) Rule base, which is a set of IF-THEN *rules*,
(ii) A bunch of *facts*, and
(iii) Some *interpreter* of the facts and rules which is a mechanism which decides which rule to apply based on the set of available facts. The interpreter also initiates the action suggested by the rule selected for application.

A Rule-base may be of the form:

R_1: If A is an animal and A barks, than A is a dog
F1: Rocky is an animal
F2: Rocky Barks

The rule-interpreter, after scanning the above rule-base may conclude: Rocky is a dog.
After this interpretation, the rule-base becomes

R_1: If A is an animal and A barks, then A is a dog
F1: Rocky is an animal
F2: Rocky Barks
F3: Rocky is a dog.

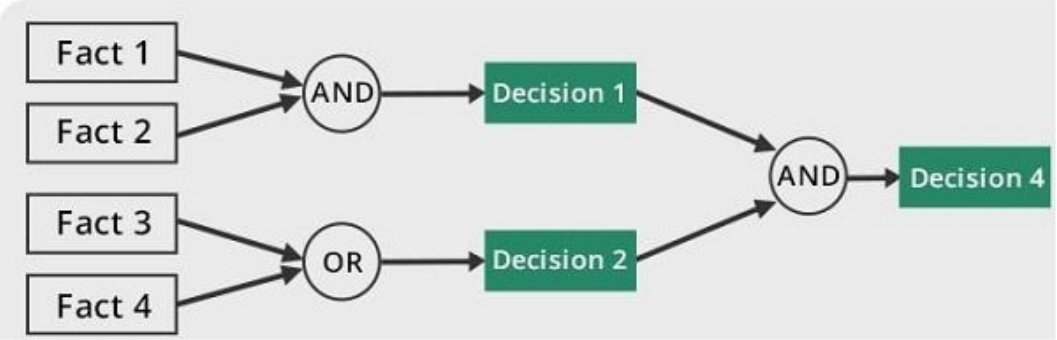
There are two broad kinds of rule-based systems:

- ***Forward chaining* systems,**
- ***Backward chaining* systems.**

FORWARD CHAINING SYSTEM:

Forward chaining is a method of reasoning in artificial intelligence in which inference rules are applied to existing data to extract additional data until an endpoint (goal) is achieved.

In this type of chaining, the inference engine starts by evaluating existing facts, derivations, and conditions before deducing new information. An endpoint (goal) is achieved through the manipulation of knowledge that exists in the knowledge base.



Forward chaining can be used in planning, monitoring, controling, and interpreting applications.

Properties of forward chaining

- The process uses a down-up approach (bottom to top).
- It starts from an initial state and uses facts to make a conclusion.
- This approach is data-driven.
- It's employed in expert systems and production rule system.

A practical example will go as follows;

Tom is running (A)

If a person is running, he will sweat (A->B)

Therefore, Tom is sweating. (B)

Advantages

- It can be used to draw multiple conclusions.
- It provides a good basis for arriving at conclusions.
- It's more flexible than backward chaining because it does not have a limitation on the data derived from it.

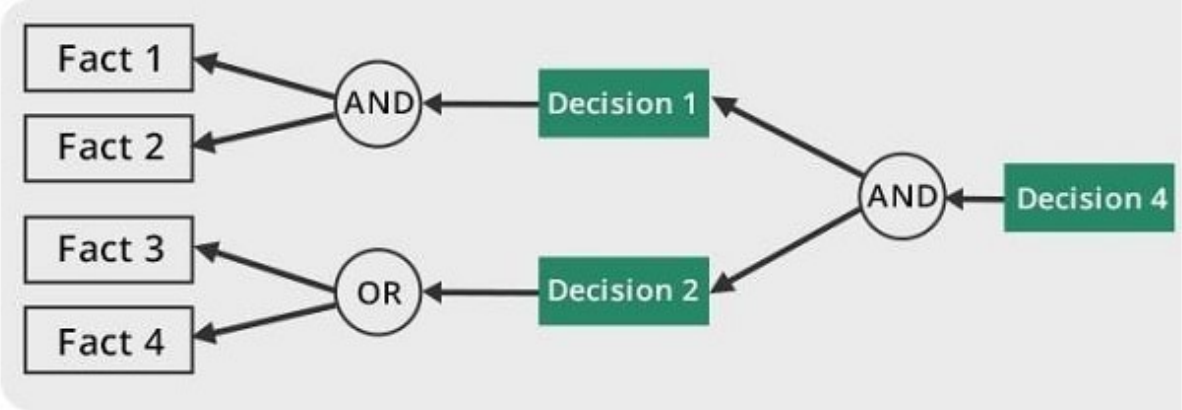
Disadvantages

- The process of forward chaining may be time-consuming. It may take a lot of time to eliminate and synchronize available data.
- Unlike backward chaining, the explanation of facts or observations for this type of chaining is not very clear. The former uses a goal-driven method that arrives at conclusions efficiently.

Backward chaining

Backward chaining is a concept in artificial intelligence that involves backtracking from the endpoint or goal to steps that led to the endpoint. This type of chaining starts from the goal and moves backward to comprehend the steps that were taken to attain this goal.

The backtracking process can also enable a person establish logical steps that can be used to find other important solutions.



Backward chaining can be used in debugging, diagnostics, and prescription applications.

Properties of backward chaining

- The process uses an up-down approach (top to bottom).
- It's a goal-driven method of reasoning.
- The endpoint (goal) is subdivided into sub-goals to prove the truth of facts.
- A backward chaining algorithm is employed in inference engines, game theories, and complex database systems.
- The modus ponens inference rule is used as the basis for the backward chaining process. This rule states that if both the conditional statement (p->q) and the antecedent (p) are true, then we can infer the subsequent (q).

A practical example of backward chaining will go as follows:

Tom is sweating (B).

If a person is running, he will sweat (A->B).

Tom is running (A).

Advantages

- The result is already known, which makes it easy to deduce inferences.
- It's a quicker method of reasoning than forward chaining because the endpoint is available.
- In this type of chaining, correct solutions can be derived effectively if pre-determined rules are met by the inference engine.

Disadvantages

- The process of reasoning can only start if the endpoint is known.
- It doesn't deduce multiple solutions or answers.
- It only derives data that is needed, which makes it less flexible than forward chaining.

6.3 SEMANTIC NETS

Semantic Network representations provide a **structured knowledge representation**. In such a network, parts of knowledge are clustered into semantic groups. In semantic networks, the concepts and entities/objects of the problem domain are represented by nodes and relationships between these entities are shown by arrows, generally, by directed arrows. In view of the fact that semantic network **representation is a pictorial depiction** of objects, their attributes and the relationships that exist between these objects and other entities. A semantic net is just a graph, where the nodes in the graph represent concepts, and the arcs are labeled and represent binary relationships between concepts. These networks provide a more natural way, as compared to other representation schemes, for mapping to and from a natural language.

For example, the fact (a piece of knowledge): ***Mohan struck Nita in the garden with a sharp knife last week,*** is represented by the semantic network shown in *Figure 1.1*.

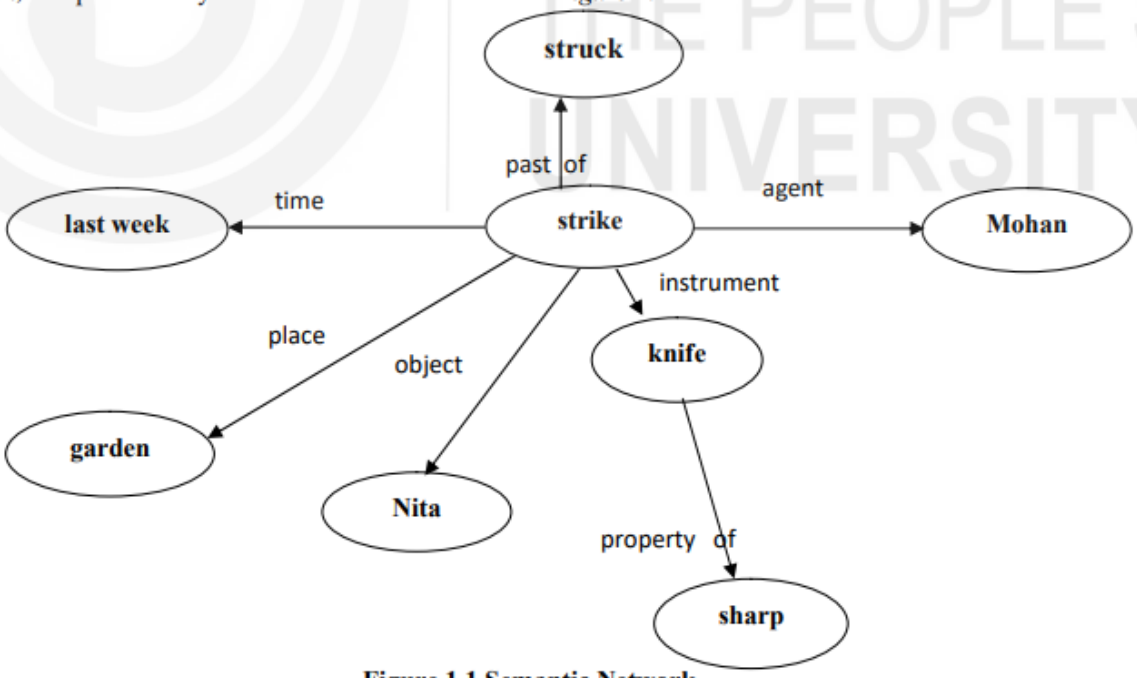


Figure 1.1 Semantic Network

6.4 FRAMES

Frames are a variant of semantic networks that are one of the popular ways of representing non-procedural knowledge in an expert system. In a frame, all the information relevant to a particular concept is stored in a single complex entity, called a frame. Frames look like the data structure, record. Frames support inheritance. They are often used to capture knowledge about *typical* objects or events, such as a car, or even a mathematical object like rectangle. As mentioned earlier, a frame is a structured object and different names like *Schema*, *Script*, *Prototype*, and even *Object* are used in stead of frame, in computer science literature.

We may represent some knowledge about a lion in frames as follows:

Mammal :
Subclass : Animal
warm_blooded : yes

Lion :
subclass : Mammal
eating-habbit : carnivorous
size : medium

Raja :
instance : Lion
colour : dull-Yellow
owner : Amar Circus

Sheru :
instance : Lion
size : small

6.5 SCRIPTS

A script is a structured representation describing a stereotyped sequence of events in a particular context.

Scripts are used in natural language understanding systems to organize a knowledge base in terms of the situations that the system should understand. Scripts use a frame-like structure to represent the commonly occurring experience like going to the movies eating in a restaurant, shopping in a supermarket, or visiting an ophthalmologist.

Thus, a script is a structure that prescribes a set of circumstances that could be expected to follow on from one another.

Scripts are beneficial because:

- Events tend to occur in known runs or patterns.
- A casual relationship between events exist.
- An entry condition exists which allows an event to take place.
- Prerequisites exist upon events taking place.

Components of a script

The components of a script include:

- **Entry condition:** These are basic condition which must be fulfilled before events in the script can occur.
- **Results:** Condition that will be true after events in script occurred.
- **Props:** Slots representing objects involved in events
- **Roles:** These are the actions that the individual participants perform.
- **Track:** Variations on the script. Different tracks may share components of the same scripts.
- **Scenes:** The sequence of events that occur.

Advantages of Scripts

- Ability to predict events.
- A single coherent interpretation maybe builds up from a collection of observations.

Disadvantages of Scripts

- Less general than frames.
- May not be suitable to represent all kinds of knowledge

Write Bayes' theorem and elaborate the meaning of each component in it.

7.4 INTRODUCTION TO BAYESIAN THEORY

Bayes' theorem is widely used to calculate the conditional probabilities of events without a joint probability. It is also used to calculate the conditional probability where intuition fails. In simple terms, probability of a given hypothesis H conditional on E can be defined as P_E (H) = P(H \& E)/ P(E), where P(E) > 0, and the term P(H \& E) also exists. Here P_E is referred to as a probability function. To simply understand the Bayes' theorem, have a look at the following definitions.

Joint Probability: This refers to the probability of two or more events simultaneously occurring, e.g. P(A and B) or P(A, B).

Marginal Probability: It is the probability of an event occurring irrespective of outcome of the other random variables e.g. P(A).

Conditional probability: A conditional probability is defined as the probability of occurrence of an event provided that another event has occurred. e.g. P(A | B).

The conditional probability can also be written in terms of joint probability as P(A|B) = P(A, B)/ P(B). In other way, if one conditional probability is given, other can be calculated as P(A|B) = P(B|A)*P(A) /P(B).

Let 'S' be a sample space in consideration. Let events 'A1' , 'A2' , ... 'An' is the set of mutually exclusive events in sample space 'S'. Let 'B' be an event from sample space 'S' provided P(B) > 0, then according to Bayes' theorem.

$P(A_k \mid B) = P(A_k \cap B) / P(A_1 \cap B)+P(A_2 \cap B) \ldots\ldots\ldots\ldots P(A_n \cap B)$, this can also be written in terms of Bayes' theorem.

$P(A_k \mid B) = P(A_k)P(B \mid A_k) / P(A_1).P(B \mid A_1)+P(A_2).P(B \mid A_2) \ldots\ldots\ldots\ldots .P(A_n).P(B \mid A_n)$

BAYE'S NETWORKS

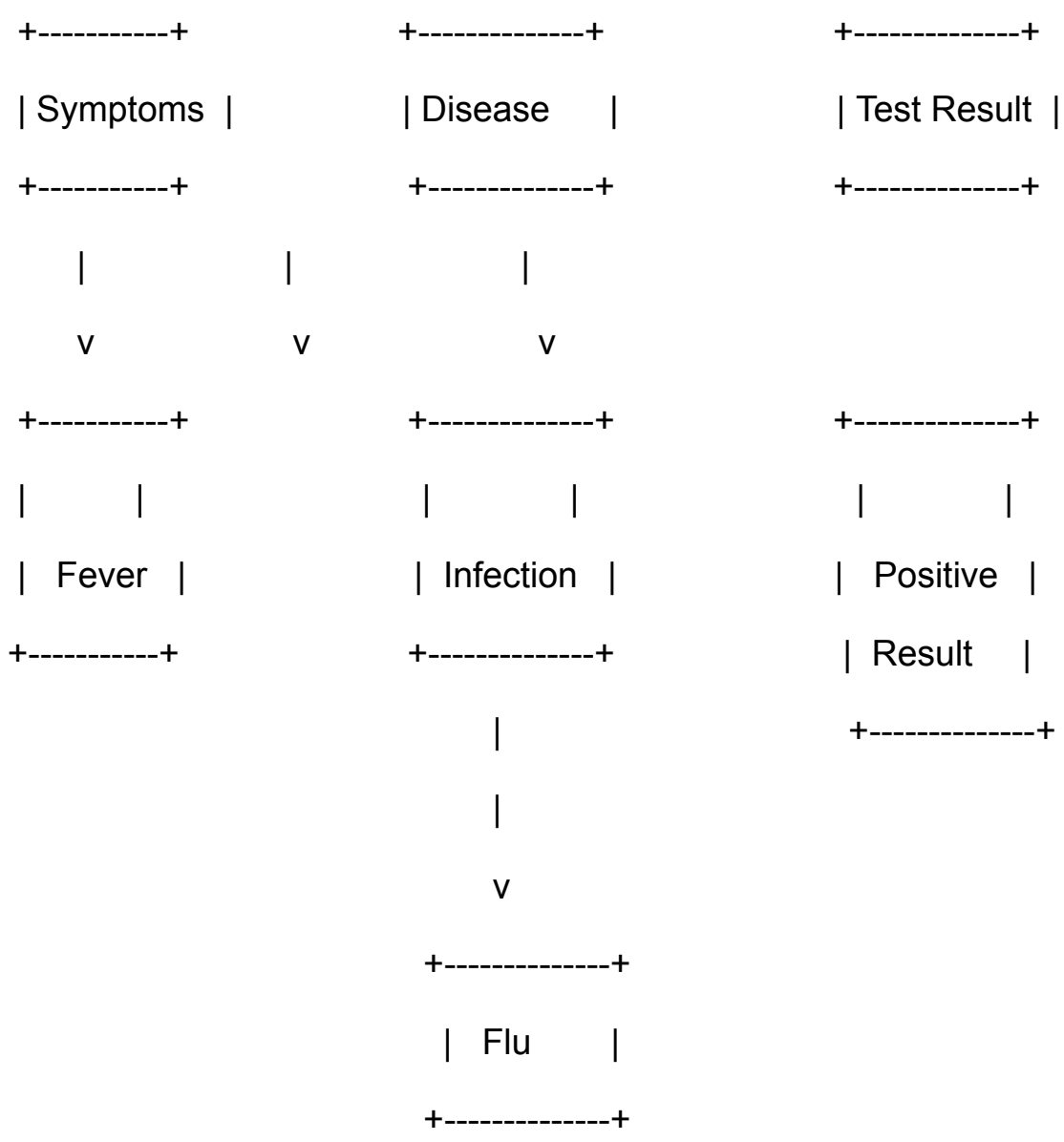
A Bayesian Network, also known as a Bayesian Belief Network, is a graphical model that represents probabilistic relationships among a set of variables. It uses a directed acyclic graph (DAG) to illustrate the dependencies between variables and conditional probability tables (CPTs) to quantify these dependencies.

Components of a Bayesian Network:

1. **Nodes:** Represent variables in the system.
2. **Edges:** Arrows connecting nodes, indicating probabilistic dependencies.
3. **Conditional Probability Tables (CPTs):** Tables associated with each node, detailing the probability of a node given its parent nodes.

Example: Medical Diagnosis Bayesian Network

Consider a Bayesian Network for medical diagnosis with three variables: `Symptoms` (S), `Disease` (D), and `Test Result` (T).



7.9 DEMPSTER SCHEFFER THEORY

Let us now discuss a mathematical theory based only on the evidence, known as Dempster-Schafer (D-S) theory given by Dempster and extended by Shafer in "Mathematical Theory of Evidences". This uses a belief function to combine separate and independent evidence pieces to quantify the belief in a statement. The D-S theory is a generalization of Bayesian probability theory where multiple possible events are assigned probabilities opposed to mutually exclusive singletons. The D-S theory assumes the existence of ignorance in knowledge creating uncertainty which in turn induces belief. Here, uncertainty of the hypothesis is represented by the belief function. The main characteristic of the theory is:

1. Multiple possible events are permitted to assign probabilities.
2. These events should be exhaustive and exclusive.

Here, the multiple sources of information are assigned some degree of belief and then aggregated using the D-S combination rule. This also limits the theory for intensive computation because of the lack of independent assumptions from such a large number of information sources.

Fuzzy Set

Let X be the set on which fuzzy sets are to be defined, e.g.,

X = {Mohan, Sohan, John, Abdul, Abrahm}.

Then X is called the ***Universal Set.***

Note: *In every fuzzy set, all the elements of X with their corresponding memberships values from 0 to 1, appear.*

(i) Degree of Membership: In respect of fuzzy sets, we do not speak of just '*membership*', but speak of '*degree of membership*'.

In the set

A = {Mohan/.2; Sohan/1; John/.7; Abrahm/.4},

Degree (Mohan) = .2, degree (John) =.4

For (ii) Equality of Fuzzy sets: Let A, B and C be fuzzy sets defined on X as follows:

Let A = {Mohan/.2; Sohan/1; John/.7; Abrahm/.4}

B = {Abrahm/.4, Mohan/.2; Sohan/1; John/.7}.

Then, as degrees of each element in the two sets, equal; we say fuzzy set A equals fuzzy set B, denoted as A = B

However, if C = {Abrahm/.2, Mohan/.4; Sohan/1; John/.7}, then

$A \neq C$.

The **Intersection C of two fuzzy sets A and B is the fuzzy set in which**, the degree of an element of C is equal to the **MINIMUM** of degrees in the two fuzzy sets.

Example:

A = {Mohan/.85; Sohan/.4; John/.6; Abdul/1; Abrahm/0}

B = {Mohan/.75; Sohan/.6; John/0; Abdul/.8; Abrahm/.3}

Then

$A \cup B$ = {Mohan/.85; Sohan/.6; John/.6; Abdul/1; Abrahm /.3}

$A \cap B$ = {Mohan/.75; Sohan/.4; John/0; Abdul/.8; Abrahm/0}

---

and, the complement of A denoted by A′ is given by

C′ = {Mohan/.15; Sohan/.6; John/.4; Abdul/0; Abrahm /1}

Ex. 2: For the following fuzzy sets A = {a/.5, b/.6, c/.3, d/0, e/.9} and B = { a/.3, b/.7, c/.6, d/.3, e/.6}, find the fuzzy sets $A \cap B$, $A \cup B$ and $(A \cap B)'$

Solution : $A \cap B = \{a/.3, b/.6, c/.3, d/0, e/.6\}$,

where degree (x in $A \cap B$) = min { degree (x in A), degree (x in B)}.

$A \cup B = \{a/.5, b/.7, c/.6, d/.3, e/.9\}$,

where degree (x in $A \cup B$) = max {degree (x in A), degree (x in B)}.

The fuzzy set $(A \cap B)'$ is obtained from $A \cap B$, by the rule:

degree (x in $(A \cap B)'$) = 1 − degree (x in $A \cap B$).

Hence

$(A \cap B)' = \{ a/.7, b/.4, c/.7, d/1, e/.4\}$

Properties of Union, Intersection and Complement of Fuzzy Sets:

The following properties which hold for ordinary sets, also, hold for fuzzy sets

Commutativity

(i) $A \cup B = B \cup A$

(ii) $A \cap B = B \cap A$

We prove only (i) above just to explain, how the involved equalities, may be proved in general.

Let $U = \{x_1, x_2 \ldots\ldots x_n\}$. *be universe for fuzzy sets A and B*

If $y \in A \cup B$, *then y is of the form {xi/di} for some i*

$y \in A \cup B \Rightarrow y = \{xi/ei\}$ *as member of A and*

$y = (xi/fi\}$ *as member of B and*

$di = max\ \{ei, fi\} = max\ \{fi, ei\}$

$\Rightarrow y \in B \cup A.$

Rest of the properties are stated without proof.

Associativity

(i) $(A \cup B) \cup C = A \cup (B \cup C)$

(ii) $(A \cap B) \cap C = A \cap (B \cap C)$

Distributivity

(i) $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$

(ii) $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$

DeMorgan's Laws

$(A \cup B)' = A' \cap B'$

$(A \cap B)' = A' \cup B'$

Involution or Double Complement

$(A')' = A$

Idempotence

$A \cap A = A$

$A \cup A = A$

Identity

$A \cup U = U \quad A \cup U = A$

$A \cap \phi = A \quad \phi \cap A = \phi$

where ϕ: empty fuzzy set = {x/0 with $x \in U$} and U: universe = {x/1 with $x \in U$}

What is Ensemble Learning ? Briefly discuss any one of the ensemble learning method.

9.6 ENSEMBLE METHODS

Ensemble learning is a general meta approach to machine learning that combines predictions from different models to improve predictive performance.

Although you can create an apparently infinite number of ensembles for any predictive modelling problem, the subject of ensemble learning is dominated by three methods. Bagging, stacking, and boosting. They are the three primary classes of ensemble learning methods, and it's essential to understand each one thoroughly.

- **Bagging Ensemble learning** is the process of fitting multiple decision trees to various samples of the same dataset and averaging the results.
- **Stacking Ensemble learning** is fitting multiple types of models to the same data and then using another model to learn how to combine the predictions in the best way possible.
- **Boosting Ensemble Learning** entails successively adding ensemble members that correct prior model predictions and produce a weighted average of the predictions.

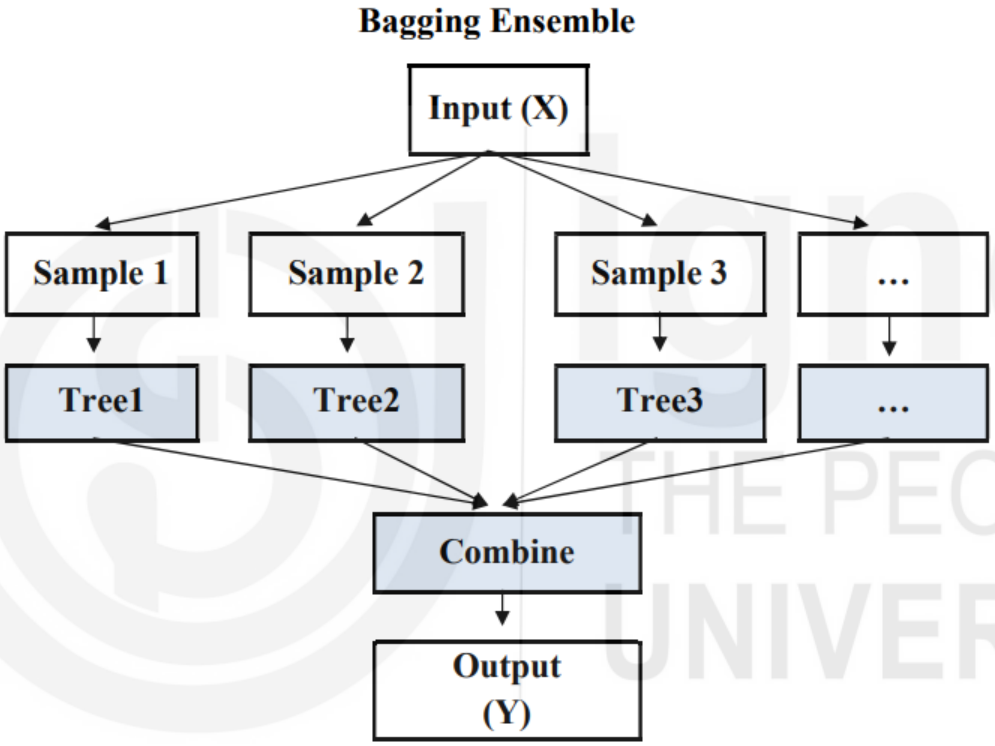
Now let's discuss each of the learning method in some detail

(I) Bagging Ensemble learning Bagging ensemble learning involves fitting numerous decision trees to various samples of the same dataset, and then averaging the results of those tree fittings to provide a final prediction.

The following is a concise summary of the most important aspects of bagging:

- Take samples of the training dataset using bootstrapping.
- Unpruned decision trees fit on each sample.
- Voting or taking the average of all the predictions.

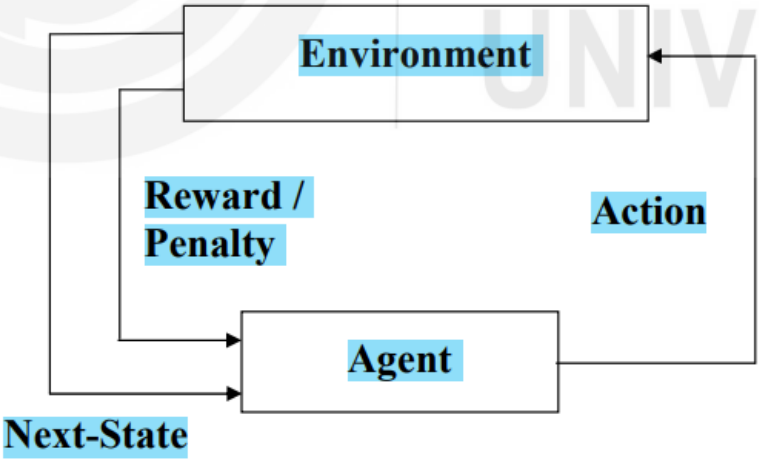
In a nutshell, bagging has an effect because it modifies the training data that is used to fit each individual member of the ensemble. This results in skillful but unique models.



Bagging ensemble learning

It is a comprehensive strategy that is simple to expand upon. For instance, additional alterations can be made to the dataset that was used for training, the method that was used to fit the training data can be modified, and the manner in which predictions are constructed can be altered.

Reinforcement Learning (RL) refers to a kind of Machine Learning method in which the agent receives a delayed reward in the next time step to evaluate its previous action. It was mostly used in games. Typically, a RL setup is composed of two components, an agent and an environment.



The following are the meanings of the different parts of reinforcement learning:

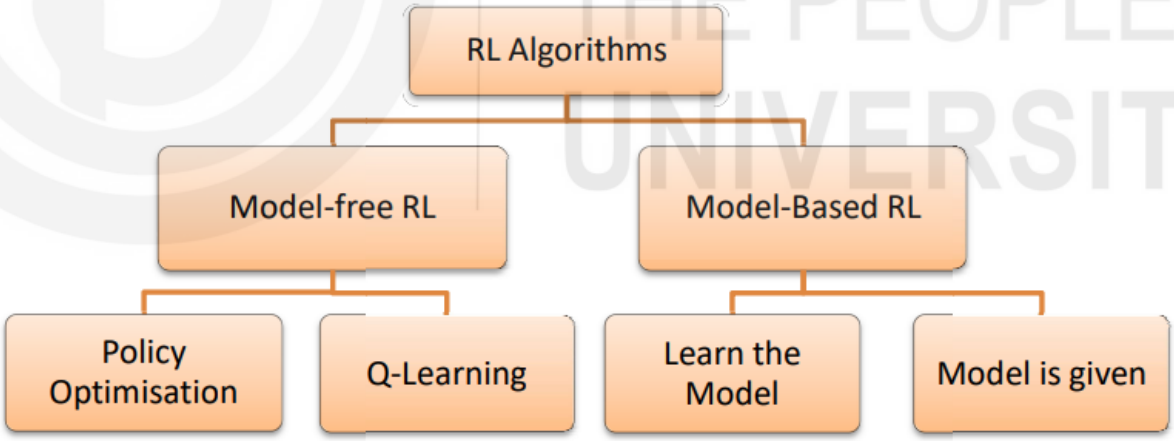
1. AGENT : The agent is the person who learns and makes decisions.

2. ENVIRONMENT : The agent's environment is where it learns and decides what to do.

3.ACTION : A group of things that the agent can do.

4. STATE : How the agent is doing in its environment.

5. REWARD : The environment gives the agent a reward for each action they choose. Usually a scalar value.



The algorithms for reinforcement learning may be broken down into two broad categories: model-free and model-based. In this section, we will analyse the key differences between these two types of reinforcement learning algorithms.

Model-Free Vs Model Based RL: The model is used to perform a simulation of the dynamic processes that take place in the environment. In other words, the model learns the transition

Model-Free vs Model-Based RL:

Model-Free RL:

1. **Definition:** Model-Free RL involves learning a policy or value function directly without building an explicit model of the environment.
2. **Components:**
   - **State (S):** Represents the current situation or configuration.
   - **Action (A):** The set of possible moves the agent can make.
   - **Reward (R):** Immediate feedback from the environment after taking an action.
   - **Policy:** The agent's strategy for selecting actions.
   - **Value Function:** Maps states to expected cumulative rewards.
   - **Function Approximator:** Methods like neural networks for approximating complex functions.

3. **Algorithms:**
    - Q-Learning: Updates action values based on temporal differences.
    - Deep Q Networks (DQN): Extends Q-Learning to handle complex state spaces.
    - Policy Optimization: Directly learns the policy through gradient ascent.
    - Actor-Critic Methods: Combines policy and value function learning.

Model-Based RL:

1. **Definition:** Model-Based RL involves building a model of the environment to simulate and plan future actions.
2. **Key Element:**
    - **Model:** Represents the environment's dynamics, providing transition probabilities between states given actions.
3. **Pros and Cons:**
    - **Pros:** Efficient in small state spaces, explicit understanding of the environment.
    - **Cons:** Less practical as state and action spaces grow, requires accurate modeling.
4. **Comparison:**
    - **Model-Free:** Learns from trial and error without an explicit environment model.
    - **Model-Based:** Learns a model of the environment to simulate and plan actions.
5. **Example:**
    - **Model-Based Algorithm:** Learns the transition probability $T(s_1|(s_0, a))$ from the present state s_0 and action a to the next state s_1.
    - **Model-Free Algorithm:** Acquires new i mation through iterative trial and error.

A brief discussion of the main types of machine learning algorithms, is given below : .

- **Bayesian:** Regardless of what the data shows, data scientists can use Bayesian algorithms to save their ideas about how models should look. Given how much attention is devoted to how the data shapes the model, you might ask why anyone would be interested in Bayesian algorithms. When you don't have much data to work with, Bayesian techniques come in handy.

  If you already knew something about a part of the model and could code that part directly, a Bayesian algorithm might make sense. Consider a medical imaging system that looks for signs of lung disease. These estimates can be incorporated into the model if a study published in a journal calculates the likelihood of various lung diseases based on a person's lifestyle.

- **Clustering :** Clustering is an easy-to-understand approach. Objects with comparable properties are combined (in a cluster). A cluster's contents are more similar than those of other clusters. Because the data are not labelled, clustering is a sort of unsupervised learning. Based on the parameters, the algorithm determines what each item is made of and assigns it to the appropriate group.

- **Decision tree :** Decision tree algorithms show what will happen when a choice is made by using a structure with branches. Decision trees can be used to show all the possible outcomes of a choice. A decision tree shows all the possible outcomes at each branch. The likelihood of the outcome is shown as a percentage for each node.

  Sometimes, online sales use decision trees. You might want to figure out who is most likely to use a 50% off coupon before sending it to them. Customers can be split into four groups:

  a) Customers who are likely to use the code if they get a personal message.
  b) Customers who will buy no matter what.
  c) Customers who will never buy.
  d) Customers who are likely to be upset if someone tries to reach out to them.

  If you send out a campaign, it's obvious that you don't want to send items to three of the groups since they will either ignore them or respond negatively. You'll get the best return on investment (ROI) if you go after the convenience.

  A decision tree will assist you in identifying these four client categories and organizing prospects and customers according to who will respond best to the marketing campaign.

- **Dimensionality reduction :** Dimensionality reduction allows systems to eliminate redundant data. These approaches remove data that is redundant, outliers, or otherwise useless. Sensor data and other IoT use cases can benefit from dimensionality reduction. The status of a sensor in an IoT system can be communicated using thousands of data points. Storing and analyzing "on" data is wasteful and wastes storage. Furthermore, minimizing redundant data increases machine

learning system performance. Finally, data visualization is also benefited from dimensionality reduction.

Dimension reduction can be accomplished in two ways :

- **Feature selection:** During this approach, a subset of the complete set of variables is selected; as a result, the number of conditions that can be utilised to illustrate the issue is narrowed down. It's normally done in one of three ways.:
  - Filter method
  - Wrapper method
  - Embedded method

- **Feature extraction:** It takes data from a space with many dimensions and transforms it into another environment with fewer dimensions.

- **Neural networks and deep learning :** A neural network is an artificial intelligence system that attempts to solve problems in the same way that the human brain does. This is accomplished by the utilisation of many layers of interconnected units that acquire knowledge from data and infer linkages. In a neural network, the layers can be connected to one another in various ways. When referring to the process of learning that takes place within a neural network with multiple hidden layers, the term "deep learning" is frequently used. Models built with neural networks are able to adapt to new information and gain knowledge from it. Neural networks are frequently utilised in situations in which the data in question is not tagged or is not organised in a particular fashion. The field of computer vision is quickly becoming one of the most important applications for neural networks. Today, one can find applications for deep learning in a diverse range of contexts.

  The process of deep learning is utilised to assist self-driving autos in figuring out what is going on in their surroundings. Deep learning algorithms analyse the unstructured data that is being collected by the cameras as they capture pictures of the environment around them. This allows the system to make judgments in what is essentially real time. The apps that radiologists use to better analyse medical images also include deep learning as an integral part of their design.

- **Linear regression :** Regression algorithms are important in machine learning and are often used for statistical analysis. Regression algorithms help analysts figure out how data points are related.

  Regression algorithms can measure how strongly two variables in a set of data are linked to each other. Regression analysis can also be used to predict the values of data in the future based on their past values. But it's important to remember that regression analysis is based on the idea that correlation means cause. Regression analysis can lead to wrong conclusions if you don't understand the context of the data.

- **Regularization to avoid over-fitting** : The process of regularisation involves modifying models in such a way that they no longer fit too well into the data . Any model that is used for machine learning can benefit from using regularisation. For example, you can regularise a decision tree

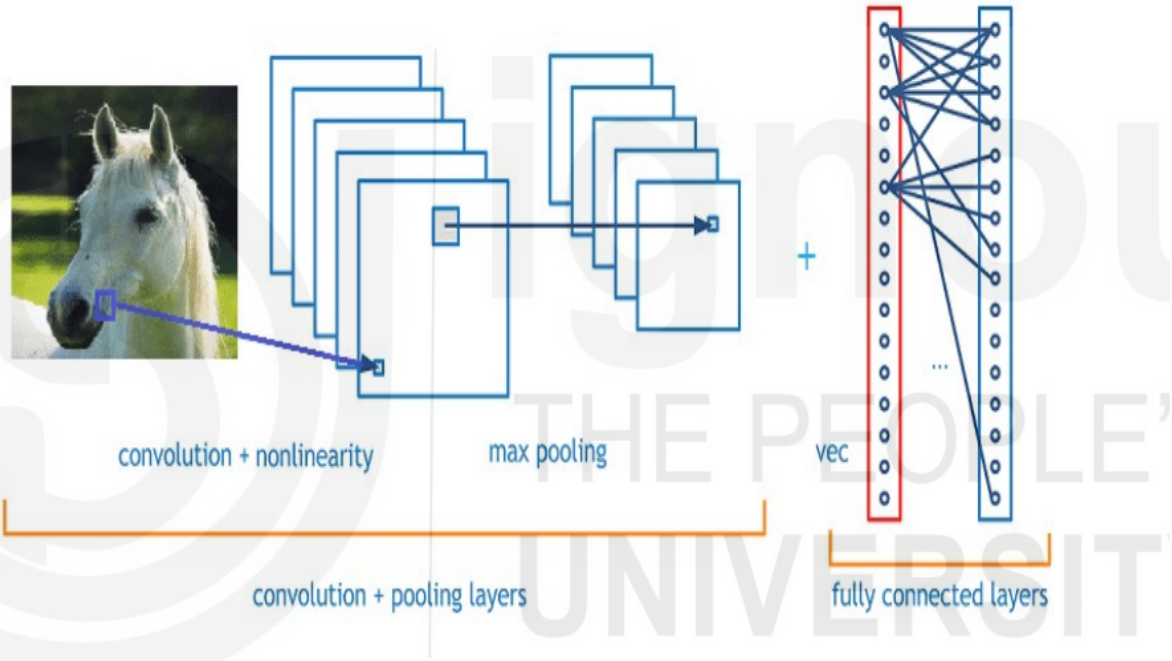
model. Models that are excessively intricate and have a tendency to be overfit can become easier to grasp with the help of regularisation. A model that has been overfit to the available data will produce accurate predictions when additional data sets are added to it. When a model is developed for a particular set of data, but that model is unable to produce accurate predictions when applied to a more general set of data, this is an example of overfitting.

- **Rule-based machine learning :** Rule-based machine learning algorithms describe data with the help of rules about relationships. A rule-based system is different from a machine learning system, which builds a model that can be used on all the data. Rule-based systems are easy to understand in general: if X data is put in, do Y. A rule-based approach to machine learning, on the other hand, can get very complicated as systems get more complicated. For example, a system might have 100 rules that are already set. As the system gets more and more data and learns how to use it, it is likely that hundreds of rules will be broken. When making a rule-based approach, it's important to make sure it doesn't get so complicated that it stops being clear.

Recurrent Neural Networks (RNN) are utilised in time series forecasting because they are ideal for time-related data. They employ some type of feedback, in which the output is fed back into the input. You can think of it as a loop that passes data back to the network from the output to the input. As a result, they are able to recall previous data and use it to make predictions.

Researchers have transformed the original neuron into more complicated structures such as GRU units and LSTM Units to improve performance. Language translation, speech production, and text to speech synthesis have all employed LSTM units extensively in natural language processing.

Convolutional Neural Networks (CNN) The term "convolution" refers to the function that is utilised by convolutional neural networks (CNN). The idea that undelies them is that rather than linking each neuron with all of the ones that come after it, we just connect it with a select few of those that come after it (the receptive field). They strive to regularise feed-forward networks in order to avoid overfitting, which is when the model is unable to generalise its findings since it can only learn from the data it has already seen. Because of this, they are particularly skilled at determining how the data are related to one another spatially. As a result, computer vision is their primary application, which includes image classification, video identification, medical image analysis, and self-driving automobiles. These are the types of tasks where they achieve near-superhuman results.



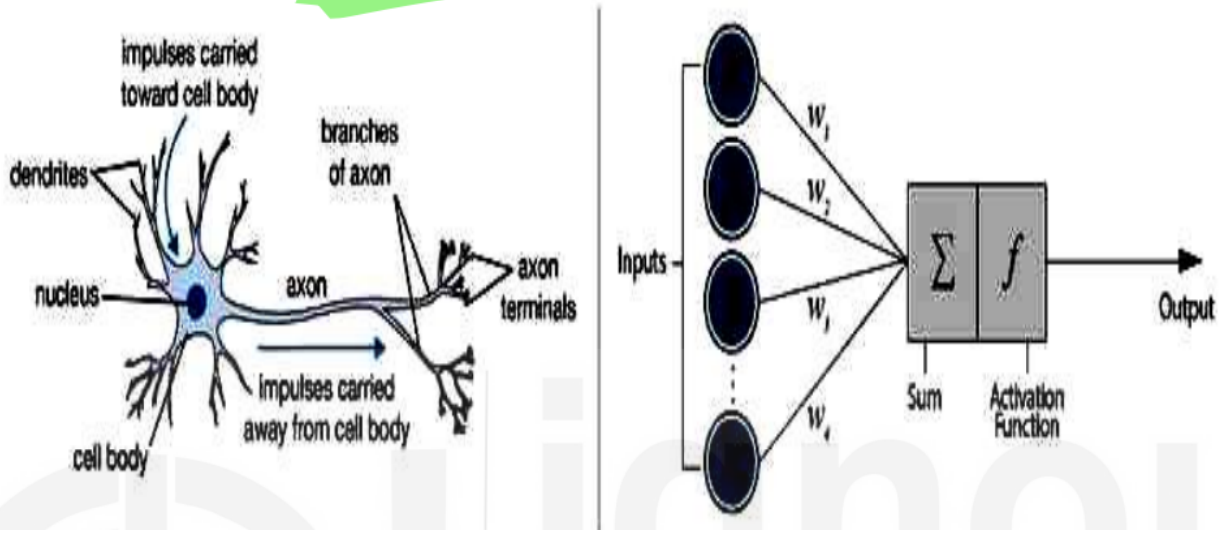
Face Recognition Based on Convolutional Neural Network

Recursive Neural Networks : Another type of recurrent network is the recursive neural network, which is set up in a tree-like manner. As a result, they can simulate the hierarchical structures training dataset's.

They're frequently utilised in NLP applications like audio-to-text transcription and sentiment analysis because they're related to binary trees, contexts, and natural-language-based parsers. They are, however, typically much slower than Recurrent Networks.

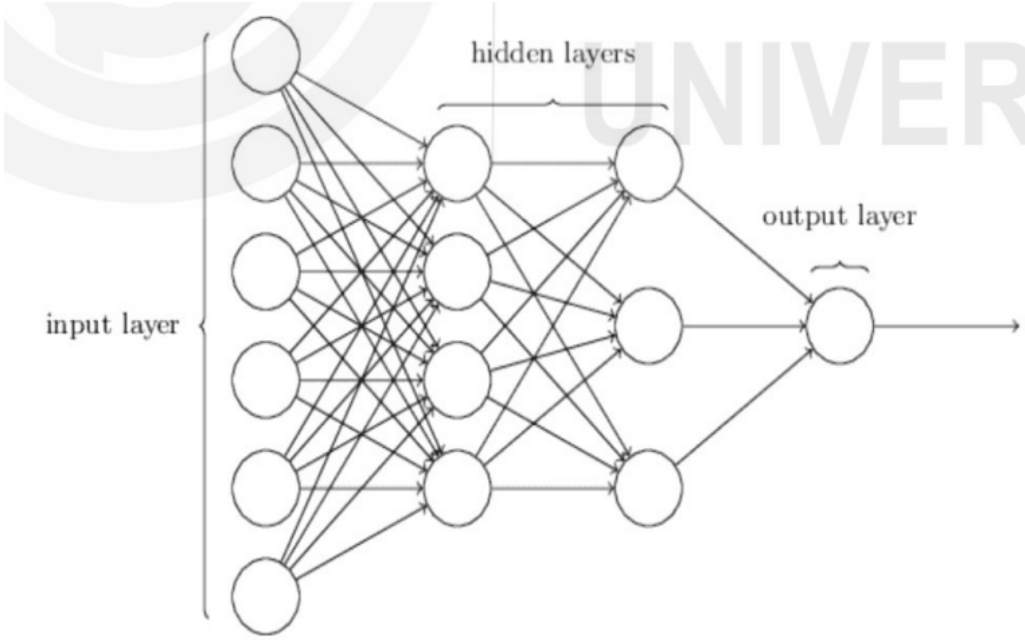
Neural Networks: Just like the human brain, Neural Networks consist of Neurons. Each Neuron takes in signals as input, multiplies them by weights, adds them together, and then applies a non-linear function. These neurons are arranged in layers and stacked close to each other.

Biological Neuron versus Artificial Neural Network



Neural Networks have proven to be effective function approximators. We can presume that every behaviour and system can be represented mathematically at some point (sometimes an incredible complex one). If we can find that function, we will know everything there is to know about the system. However, locating the function can be difficult. As a result, we must use Neural Networks to estimate it.

Feed-forward neural networks (FNN) : Typically, feed-forward neural networks, also known as FNN, are completely connected, this implies that each neuron in one layer is connected to each neuron in the layer next to it. A "Multilayer Perceptron" is the name given to the structure shown below and that is the topic of discussion here. A multilayer perceptron, has the ability to learn associations between the data that are not linear, in contrast to a single-layer perceptron, which can only learn patterns that can be separated in a linear manner. FNN are exceptionally well on tasks like classification and regression. Contrary to other machine learning algorithms, they don't converge so easily. The more data they have, the higher their accuracy.



Restricted Boltzmann Machines (RBM) are stochastic neural networks that can learn a probability distribution over their inputs and so have generative capabilities. They differ from other networks in that they only have input and hidden layers (no outputs).

They take the input and create a representation of it in the forward phase of the training. They rebuild the original input from the representation in the backward pass. (This is similar to autoencoders, but in a single network.)

Several RBMs are piled on top of each other to form a Deep Belief Network. They have the same appearance as Fully Connected layers, but they are trained differently.

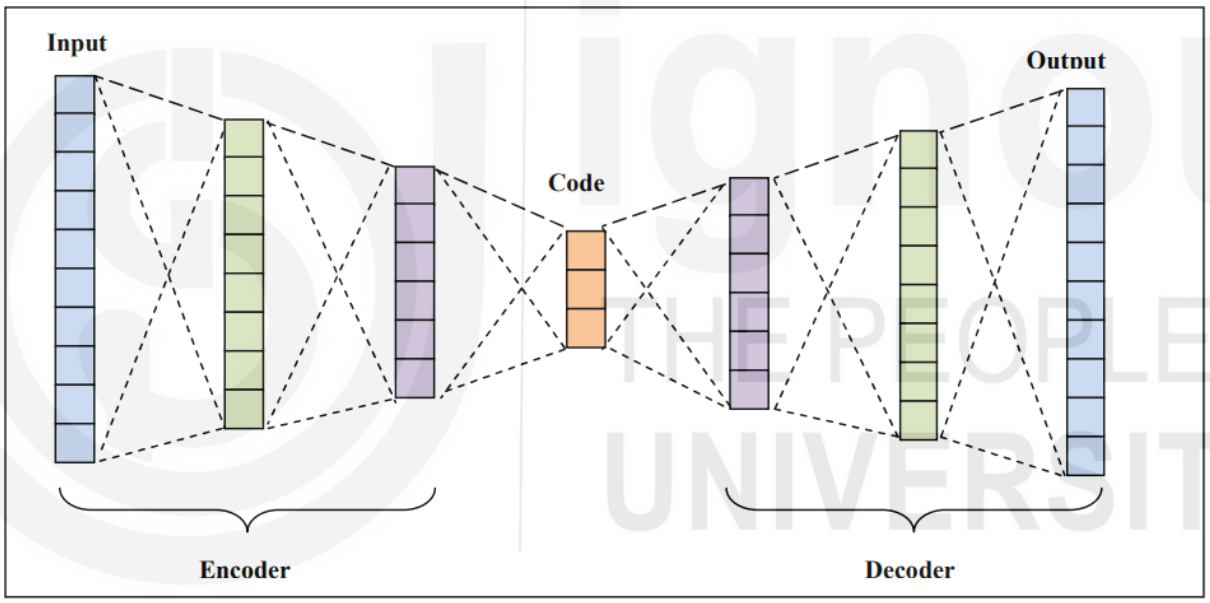
Transformers are also very new, and they are mostly employed in language applications because recurrent networks are becoming obsolete. They are based on the concept of "attention," which instructs the network to focus on a certain data piece.

Instead of complicating LSTM units, you may use Attention mechanisms to assign varying weights to different regions of the input based on their importance. The attention mechanism is simply another weighted layer whose sole purpose is to change the weights such that some parts of the inputs are given greater weight than others.

In actuality, transformers are made up of stacked encoders (encoder layer), stacked decoders (decoder layer), including several attention layers (self- attentions and encoder-decoder attentions)

Auto-Encoders (Auto Encoder Neural Networks) are a type of unsupervised technique that is used to reduce dimensionality and compress data. Their technique is to try and make the output equal to the input. They are attempting to recreate the data.

An encoder and a decoder are included in Auto-Encoders. The encoder receives the input and encodes it in a lower-dimensional latent space. Whereas, the decoder is used to decode that vector back to the original input.



Generative Adversarial Networks (GANs): Ian Goodfellow introduced Generative Adversarial Networks (GANs) in 2016, and they are built on a basic but elegant idea: You need to create data, such as photos. What exactly do you do?

You must construct two models. You teach the first one to make up fake data (generator) and the second one to tell the difference between actual and fake data (discriminator). And you turned them against one another.

The generator develops better and better at image production, as its ultimate purpose is to mislead the discriminator. As its purpose is to avoid being tricked, the discriminator improves its ability to identify fake from real images. As a result, we now have extremely realistic fake data from the discriminator.

Video games, astronomical imagery, interior design, and fashion are all examples of Generative Adversarial Networks at action. Essentially, you can utilise GANs if you have photos in your fields. Do you recall the movie Deep Fakes? That was all created by GANs.

* *Differentiate between Machine learning and Data mining*

The process of "data mining" refers to the extraction of information from data that is latent, previously unknown, and has the potential to be beneficial. Building computer algorithms that can automatically search through large databases for recurring structures or patterns is the goal of this project. In the event that robust patterns are discovered, it is likely that they will generalise to enable accurate predictions on future data.

In the renowned book "Data Mining: Practical Machine Learning Tools and Techniques," written by Ian Witten and Eibe Frank, the subject matter is thoroughly covered. The activity known as "data mining" refers to the practise of locating patterns within data. The procedure needs to be fully automatic or, at the very least, semiautomatic. The patterns that are found have to be significant in the sense that they lead to some kind of benefit, most commonly an economic one. Consistently and in substantial amounts, the statistics are there to be found.

Machine learning, on the other hand, is the core of data mining's technical infrastructure. It is used to extract information from the raw data that is stored in databases; this information is then expressed in a form that is understandable and can be applied to a range of situations.

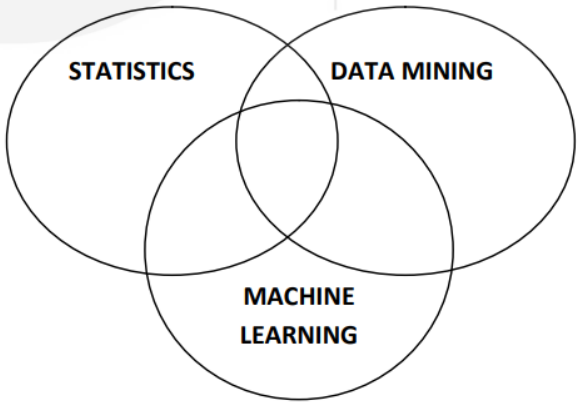


Figure 1 : Machine Learning – Data Mining - Statistics

The Machine Learning Cycle: Making a machine learning application is similar to making a machine learning algorithm work, which is an iterative process. You can't just train a model once and leave it alone, because data changes, preferences change, and new competitors come along. So, when your model goes into production, you need to keep it updated. Even though you won't need as much training as when you created the model, don't expect it to run on its own.

Figure 2 :Machine Learning Cycle at a Glance

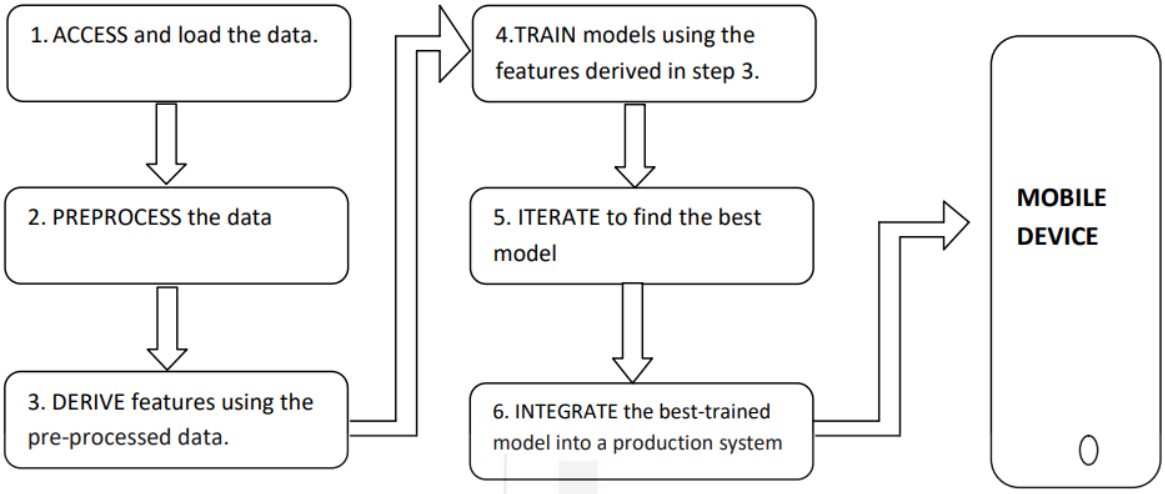


Figure 2 : Machine Learning Cycle at a Glance

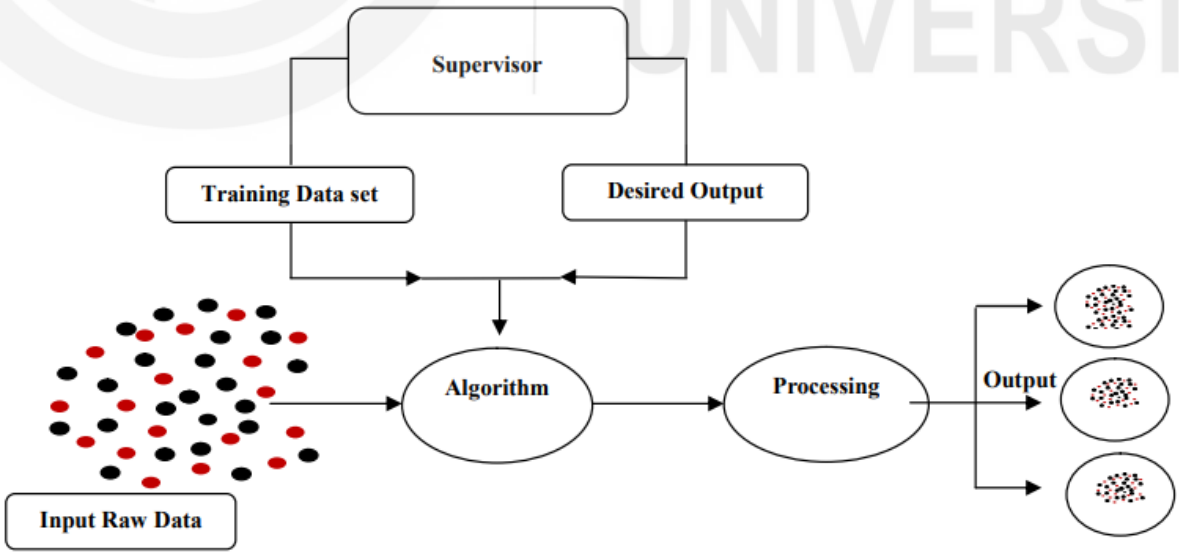
One step in the machine learning cycle is choosing the right machine learning algorithm. So, let's look at how the machine learning cycle works.

The steps in the machine learning cycle are as follows:

1. *Data Identification*
2. *Data Preparation*
3. *Selection of machine learning algorithm:*
4. *Training the algorithm to develop a model*
5. *Evaluating the model*
6. *Deploying the model*
7. *Performing Prediction*
8. *Assess the predictions*

Fundamentally Machine learning involves two classes of Learning algorithms:

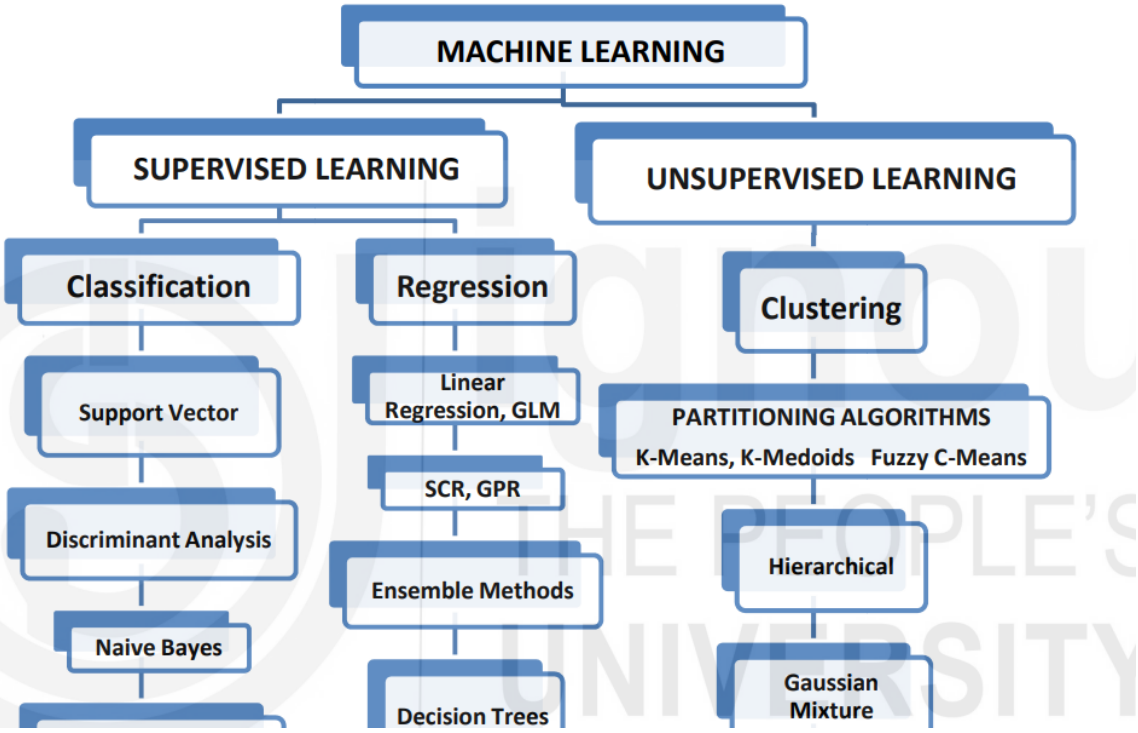
a) Supervised learning, which requires training a model with data whose inputs and outputs are already known in order for the model to be able to predict future outputs, such as whether or not an email is authentic or spam or whether or not a tumor is cancerous. Classification models classify given data into categories. Imaging for medical purposes, speech recognition, and rating credit are a few examples of typical applications.



Following Steps are Involved in Supervised Learning, and they are self explanatory:

1. Determine the Type of Training Examples
2. Prepare/Gather the Training Data
3. Determine Relation Between Input Feature & Representing Learned Function
4. Select a Learning Algorithm
5. Run the Selected Algorithm on Training Data
6. Evaluate the Accuracy of the Learned Function Using Values from Test Set

b) Unsupervised learning analyses data to uncover previously unknown patterns or structures. It is used to infer conclusions from sets of data that contain inputs but no tagged answers. The most prevalent method of learning without being observed is clustering. Exploratory data analysis is used to uncover hidden patterns or groups in data. Clustering can be used for gene sequence analysis, market research, and object recognition.



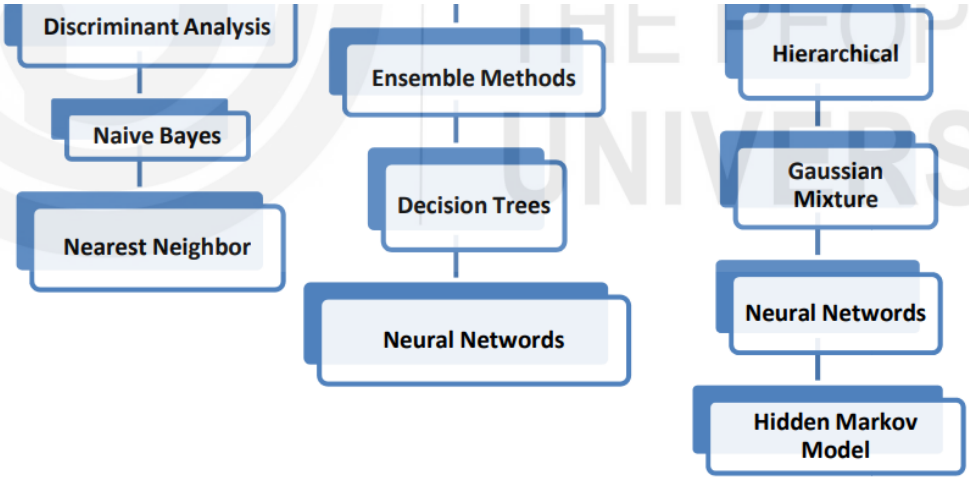


Figure – 3 Machine Learning Algorithms

Supervised Learning Techniques: Every supervised learning method may be broken down into one of two categories: classification or regression. Methods like as classification and regression, which are employed in supervised learning, are put to use in the development of models that are able to forecast the future.

- **Classification techniques** : Classification methods make predictions about discrete outcomes, such as whether an e-mail is genuine or spam or if a tumour is cancerous or not. Classification models classify incoming data into categories. Imaging for medical purposes, speech recognition, and rating credit are a few examples of typical applications.

- **Regression methods** : Predictions can be made with regression algorithms about things like shifts in temperature or alterations in the quantity of power consumed. The most typical applications are stock price predicting, handwriting recognition, electricity load forecasting, acoustic signal processing, and other similar tasks.

This section covers the following types of classification:

- **Binary classification** is a task for which there are only two possible outcomes. It means predicting which of the two classes will be correct. For example, dividing people into male and female.

Some popular algorithms that can be used to divide things into two groups are:

- Logistic Regression.
- k-Nearest Neighbors.
- Decision Trees.
- Support Vector Machine.
- Naive Bayes.

Major application areas of Binary Classification:

- Detection of Email spam.
- Prediction of Churn.
- Prediction of Purchase or Conversion(buy or not).

- **Multi-class classification.:**Multi-class classification means putting things into more than two groups. In multiclass classification, there is only one target label for each sample. This is done by predicting which of more than two classes the sample belongs to. An animal, for instance, can either be a cat or a dog, but not both. Face classification, plant species classification, and optical character recognition are some of the examples.

  Popular algorithms that can be used for multi-class classification include:

  - k-Nearest Neighbors.
  - Decision Trees.
  - Naive Bayes.
  - Random Forest.
  - Gradient Boosting.

Binary classification algorithms can be changed to work for problems with more than two classes. This is done by fitting multiple binary classification models for each class vs. all other classes (called "one-vs-rest") or one model for each pair of classes (called one-vs-one).

- **One versus the Rest**: Fit one binary classification model for each class versus all of the other classes in the dataset.
- **One-versus-one**: Fit one binary classification model for each pair of classes using the one-on-one comparison method.

Binary classification techniques such as logistic regression and support vector machine are two examples of those that are capable of using these strategies for multi-class classification.

- **Multi-label classification:** Multi-label classification, also known as more than one class classification, is a classification task in which each sample is mapped to a collection of target labels. This classification task involves making predictions about one or more classes; say for example, a news story can be about Games, People, and a Location all together at the same time.

  Note : Classification algorithms used for binary or multi-class classification cannot be used directly for multi-label classification.

Specialized versions of standard classification algorithms can be used, so-called multi-label versions of the algorithms, including:

- Multi-label Decision Trees
- Multi-label Random Forests
- Multi-label Gradient Boosting

Another approach is to use a separate classification algorithm to predict the labels for each class.

• **An imbalanced classification** is a task in which the number of examples in each class is not the same. Examples includes Fraud detection, Outlier detection, Medical diagnostic tests.

The different types of classifications discussed above, have to deal with different type of learners, and **Learners in Classification Problems are categorized into following two types :**

1. **Lazy Learners:** Lazy Learner will first store the training dataset, and then it will wait until it is given the test dataset. In the case of the Lazy learner, classification is done based on the information in the training dataset that is most relevant to the question at hand. Less time is needed for training, but more time is needed for making predictions. K-NN algorithm and Case-based reasoning are two examples.
2. **Eager Learners:** Eager Learners use a training dataset to make a classification model before they get a test dataset. Eager learners, on the other hand, spend less time on training and more time on making predictions. Decision Trees, Naive Bayes, and ANN are some examples.

Examples of eager learners include the classification techniques of Bayesian classification, decision tree induction, rule-based classification, classification by back-propagation, support vector machines, and classification based on association rule mining. Eager learners, when presented with a collection of training pairs, will construct a generalisation model (also known as a classification model) before being presented with new tuples, also known as test tuples, to classify. One way to think about the learnt model is as one that is prepared and eager to categorise tuples that have not been seen before.

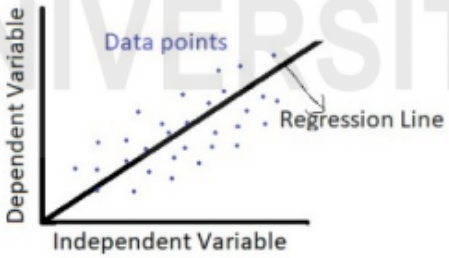
*Logistic Regression

Logistic Regression is a statistical method for predicting the probability of a binary outcome (0 or 1). Key points include the sigmoid function for probability transformation, setting a threshold for classification, and assumptions like a categorical dependent variable. Types include binomial, multinomial, and ordinal. The regression equation is derived from linear regression, and odds are crucial, calculated using the logit function (log(p/(1 – p))). Logistic Regression is widely used in machine learning for binary classification tasks due to its effectiveness in predicting outcomes.

11.3 LINEAR REGRESSION

Linear regression is a mathematical method implemented where we want to find the response variable and predictor variables. When the relationships are linear then it is called as linear regression or otherwise it is called as a nonlinear regression. Linear regression makes prediction for continuous/real or numerical variables like age,salary,price etc.

As shown in figure x-axis represent independent variable and y-axis represent dependent variable. A Line with some slope is called linear regression line which shows the relationship between the independent and dependents variable and dots represents the point of the data sets, where some points lie on the line and some other points lie above and below of the line.



Linear regression using least square method

Mathematical function is used to find the sum of squares (square of the distance of the points and the regression line) of all the data points. Least square method is a statistical method given by Carl Friedrich Gauss used to determine the best fit line or the regression line by minimizing the sum of squares. Least square method is used to find the line having minimum value of the sum of squares and this line is the best-fit regression line.

Regression line is y=m*x+c where

y= Estimated or predicted value (Dependent Variable)

x= Value of x for observation (Independent variable)

c= Intercept with the y-axis.

m= Slope of the line

Example :1

Consider the following set of data points (x,y), find the regression line for the given data points.

X	1	2	3	4	5
Y	3	4	2	4	5

Solution:

X	y	$(x - \bar{x})$	$(y - \bar{y})$	$(x - \bar{x})^2$	$(x - \bar{x})(y - \bar{y})$
1	3	-2	-0.6	4	1.2
2	4	-1	0.4	1	-0.4
3	2	0	-1.6	0	0
4	4	1	0.4	1	0.4
5	5	2	1.4	4	2.8
3	3.6	0	0	10	4

where $m=\frac{\sum(x-\bar{x})(y-\bar{y})}{\sum(x-\bar{x})^2}$

m= 4/10= 0.4

$\bar{x}$=mean of x=3

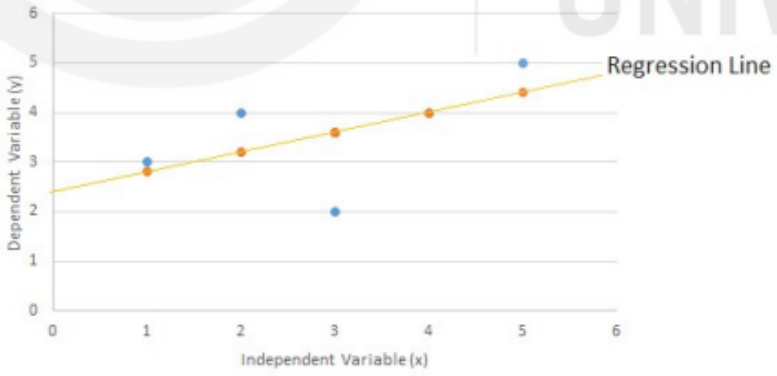
$\bar{y}$=mean of y=3.6

y=mx+c

m= 0.4

c=2.4

y=.4x+2.4



In the above figure blue points are the actual points and yellow points are the predicted points using least square method. Some points represented by blue color lie above the line while some other blue color points lie below the line. However some points represented by the yellow color lie on the line. All other points not lying on the line are the far away from the line with some distance. Thus, actual blue data

points and the predicted yellow data points contain some distance between them. This distance or difference between the data points represent an error.

Cost function is used to find the distance between the actual data point value lying other than the regression line and the predicted value of data points lying on the regression line. Cost function optimizes the regression coefficient or weights. It measures how a linear regression model is performing.
Difference between the actual value y on y-axis and predicted value $\hat{y}$ is (y-$\hat{y}$), and cost=(y-$\hat{y}$)^2
if there are n number of data points then the cost function will be

$$\text{cost} = \frac{1}{2n}\sum_{i=1}^{n}(y - \hat{y})$$

or

$$\text{cost} = \frac{1}{n}\sum|(y - \hat{y})|$$

Since cost function provide the error between the actual value and predicted value so minimizing the value of cost function will improve the prediction value. Higher the cost function value will degrade the performance.

Mean Squared Error (MSE): The average of squared of the distance measured between the actual data points lying other than the line and predicted data points lying on the line is called as a mean squared error. It is written as:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{n}\left(y_i - (a_1 x_i + a_0)\right)^2$$

Where N = total number of data points

y_i = Actual value

$(a_1 x_i + a_0)$ = Predicted value with slope a_1 and intercept a_0

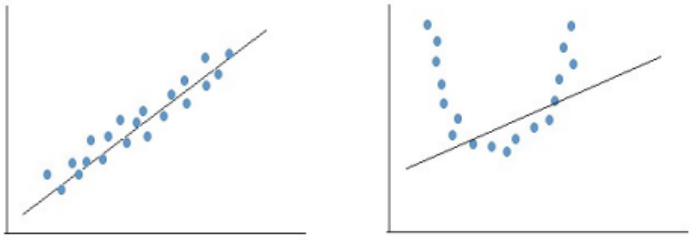
Mean Absolute Error (MAE) is used to determine by calculation sum of all errors divided by the total number of errors in a group of predictions. While considering a group of data points their directions are not important. In other words, it is a mean of absolute differences among actual value and response value results where all individual deviations have even importance.

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{m}|\hat{y}^{(i)} - y^{(i)}|$$

11.4 POLYNOMIAL REGRESSION ALGORITHM

Linear model can apply to data set having linear in nature, however if we have data set of nonlinear in nature then nonlinear model is to be applied.

As shown in figure all the data points are linear in nature. All points are close to the line, linear model regression model can be applied to the data sets. In figure 2 all the data points are nonlinear in nature so linear model cannot fit all the data points, only 2 or3 data points can be fitted to the linear model and all other points are far away from the line. Loss value for this graph will be very high and accuracy will be reduced.



1. Polynomial regression is a type of regression algorithm in which specifies the relationships between independent and dependent variable. But here the independent variables are of n^{th} degree polynomial.

2. General equation for the polynomial curve fitting

$$y = m_1x^1 + m_2x^2 + m_3x^3 + \dots\dots\dots + m_nx^n$$

$$y = \sum_{i=1}^{1=D} m_ix^i + C$$

Example 2. Consider the following set of data points (x,y). Find the 2nd order polynomial $y=a_0 + a_1 x_i + a_2 x_i^2$, and using polynomial regression determine the value of y when x is 40.

X	40	10	-20	-88	-150	-170
Y	5.89	5.99	5.98	5.54	4.3	3.33

Solution. From the given data points (x,y):

x_i	y_i	x_i^2	x_i^3	x_i^4	$x_i y_i$	$x_i^2 y_i$
40	5.89	1600	64000	2560000	235.6	9424
10	5.99	100	1000	10000	59.9	599
-20	5.98	400	-8000	160000	-119.6	2392
-88	5.54	7744	-681472	59969536	-487.52	42901.8
-150	4.3	22500	-3375000	506250000	-645	96750
-170	3.33	28900	-4913000	835210000	-566.1	96237
$\sum_{i=1}^{n} x_i$ = -378	$\sum_{i=1}^{n} y_i$ = 31.03	$\sum_{i=1}^{n} x_i^2$ = 61244	$\sum_{i=1}^{n} x_i^3$ = -8912472	$\sum_{i=1}^{n} x_i^4$ = 1404159536	$\sum_{i=1}^{n} x_i y_i$ = -1522.7	$\sum_{i=1}^{n} x_i^2 y_i$ = 248304

$$\begin{bmatrix} 6 & -378 & 61244 \\ -378 & 61244 & -8912472 \\ 61244 & -8912472 & 1404159536 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 31.03 \\ -1522.7 \\ 248304 \end{bmatrix}$$

By solving above matrix the value a_0,a_1 and a_2 will be

a_0=6.07647

a_1=-0.00253987

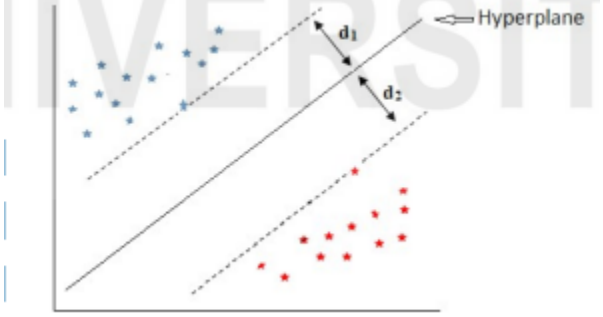
a_2=-0.000104319

$y= 6.07647 - 0.00253987\, x - 0.000104319\, x^2$

y(50)= 6.07647 − 0.00253987 X 50 - 0.000104319 X 2500

= 5.68

SUPPORT VECTOR REGRESSION



Support Vector Machines (SVM) are versatile in solving both classification and regression problems. In a classification scenario with two distinct categories, the objective is to find a hyperplane that effectively separates these categories. The hyperplane is positioned between the categories, with two marginal lines on either side.

These marginal lines are at a specific distance from the hyperplane, ensuring efficient categorization of points. Importantly, they must pass through at least one of the closest data points, termed support vectors. There can be multiple support vectors influencing the position of the marginal hyperplanes.

To achieve the optimal separation, the goal is to select the marginal hyperplane that maximizes the combined distance (d1 + d2) from the hyperplane to the marginal lines. This process involves choosing the hyperplane that ensures the maximum margin between the categories.

In cases where a linear hyperplane is insufficient for separation in the original two-dimensional space (Fig a), SVM kernels come into play. These kernels transform the data points into a higher-dimensional space, such as 3D or 4D (Fig b). In this transformed space, a hyperplane can effectively divide the data points into distinct categories, overcoming the limitations of a linear approach.

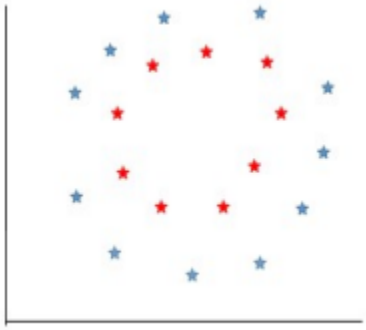


Fig a

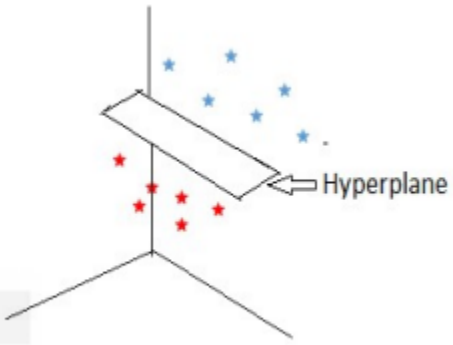


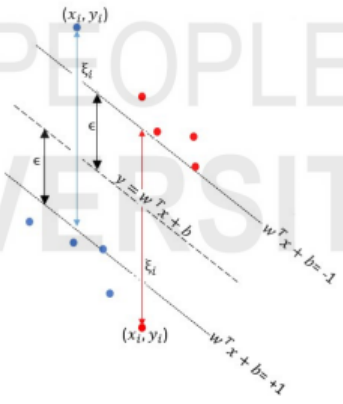
Fig b

Consider the following graph

As shown in figure equation of hyperplane is $y=w^Tx+b$, where b is constant having value zero since line is passes through the coordinate (0,0) and the slope of line m is -1.

$y=w^Tx$ (since b=0)

Any point that lies below the hyperplane (w^Tx) contribute to the positive value of x, and so is an example of the blue data points, while any data point that lie above the hyperplane (w^Tx) contribute to the negative value of x, and so is an example of the red data points.



For a given margin value M we can say for any value of x where $w^Tx \geq M$ lies on blue points, and for any value of x where $w^Tx \leq -M$ lies on red points. Now consider a point x_i^+ that lies at the positive margin of the hyperplane then $w^Tx_i^+=M$. Here x_i^+ is a support vector. On travelling to the opposite direction perpendicular to the positive margine we will reach a point closest to the negative margin of the hyperplane called as x_i^-.

If x_1 and x_2 are two negative and positive regression vector lies on marginal lines $w^Tx+b=-1$ and $w^Tx+b=+1$, the distance between marginal lines can be determined by

$$w^T(x_2 - x_1)=2$$

$$\frac{w^T(x_2-x_1)}{||w||}=\frac{2}{||w||}$$

We have to maximize $\frac{2}{||w||}$ subject to $w^T(x_2 - x_1) \geq +1$ and maximize $\frac{2}{||w||}$ subject to $w^T(x_2 - x_1) \leq -1$ for all i=1, 2…..n. In generalized form, $y_i * w^T x_i + b_i \geq +1$, and we need to minimize $\frac{||w||}{2}$ (reciprocal of $\frac{2}{||w||}$).

If all the data points are classified by the marginal line, then it will overfit the machine. And this is not happening in real scenario. It is not always possible that all the data point lies on the right side of the classification. As shown in figure one of the red data points lie below positive margin and one of the blue data points lie above negative margin of the hyperplane. These two data points lie in opposite plane area. If ξis the distance of the data point from respective marginal line, we need to find out the error $\xi_i$for such points.

$$y_i - (w^T x_i + b_i) \leq \epsilon + \xi_i \text{ for each } \xi_i \geq 0$$

$$(w^T x_i + b_i) - y_i \leq \epsilon + \xi_i$$

Error computed = $C_i \sum_{i=1}^{n} \xi_i$ where C_i is the number of error and ξ_i is the error value

Thus it is required to minimize $(w^*, b^*) = \frac{2}{||w||} + C_i \sum_{i=1}^{n} \xi_i$

Where * represent the optimal value.

Applications of Support Vector Regression

Used to solve supervised regression problems.

Can be used in both linear and non linear type of data.

Prediction of fire in forest during weather changes.

Prediction of electric power demand.

Following are various types of regression algorithms.

Linear regression: Linear regression algorithm comes into existence when there is only one dependent variable and independent variables can be be one or more in numbers. If there is a single independent variable, then it is called as simple linear regression. In linear regression the relationships between the dependent and independent variables are linear i.e. of type $y_i=a+b*x_i$; where y_i is a dependent variable and x_i is a independent variable. Variable b is the slope of the line and a is intercept with the axis. Example child height=a+b*(parent height)

Multiple Linear Regression: When there is only one dependent variable and more than one independent variables, then it results in multiple linear regression i.e. $y=a+bx_1+cx_2+dx_3$. ; example weight = a+b * (daily meal)+ c* (daily exercise)

Logistic regression: In logistic regression algorithm dependent variable is binary in nature (False/True). This algorithm is generally used under cases like testing of the medicines, to detect the bank fraud etc.We had already discussed the concept of logistic regression in unit no. 10 of this course.

Polynomial regression: Polynomial regression is described with the help of polynomial equation where the occurrence of independent variable is more than one. There is no linear relationships between the dependent and independent variables. It results in a curved line instead of a straight line i.e. $y=c+a*x+b*x^2$

***What is the difference between Regression and classification?**

ChatGPT

Regression and classification are two distinct types of supervised machine learning tasks with fundamental differences in their objectives.

Regression:

1. **Objective:** The primary goal of regression is to predict a continuous numerical output or value. It deals with predicting a quantity within a specific range.
2. **Output:** The output variable is quantitative and has a meaningful order.
3. **Example:** Predicting house prices, temperature, or stock prices are common regression tasks.

Classification:

1. **Objective:** The primary goal of classification is to assign input data into predefined categories or classes.
2. **Output:** The output variable is categorical, representing different classes or labels.
3. **Example:** Identifying whether an email is s[illegible]n or not, recognizing handwritten digits, or classifying images of animals are typical classification tasks.

While regression and classification are distinct tasks with different objectives, there are some similarities between them:

Supervised Learning:

- **Common Ground:** Both regression and classification fall under the umbrella of supervised learning. In supervised learning, the algorithms are trained on labeled data, where the input features are associated with corresponding output labels or values.

Input-Output Relationship:

- **Input Features:** In both regression and classification, the models utilize input features to make predictions or assignments.
- **Output Relationship:** They involve establishing a relationship between input features and output, whether the output is a continuous value (regression) or a categorical label (classification).

Model Training:

- **Training Process:** Both regression and classification models undergo a training process where they learn patterns and relationships from the labeled training data.

Evaluation:

- **Performance Metrics:** Similar evaluation metrics can be used for both regression and classification tasks, such as accuracy, precision, recall, and F1 score, depending on the specific problem.

Generalization:

- **Generalization:** The ultimate goal for both regression and classification models is to generalize well to new, unseen data. This involves the ability to make accurate predictions or assignments on data not er ntered during training.

(e) What is Skolomization ? Skolomize the expression : 4

$$(\exists_{X_1})(\exists_{X_2})(\forall_{Y_1})(\forall_{Y_2})(\exists_{X_3})(\forall_{Y_3})$$
$$P(X_1, X_2, X_3, Y_1, Y_2, Y_3)$$

Skolemization is a process for removing existential quantifiers from first-order logic formulas. It is often used as a preliminary step to other automated reasoning algorithms, such as theorem proving and model checking.

What is Logistic Regression? Briefly discuss the various types of logistic regressions.

Answer:

Logistic regression is a statistical method used for binary classification problems. It is a linear model that predicts the probability of a binary outcome. Logistic regression is a popular choice for binary classificatior problems because it is simple to understand, implement, and interpret.

There are three main types of logistic regression:

- **Binary logistic regression:** This is the most common type of logistic regression. It is used to predict the probability of a binary outcome, such as whether an email is spam or not spam.

- **Multinomial logistic regression:** This type of logistic regression is used to predict the probability of one of three or more outcomes. For example, multinomial logistic regression can be used to predict the probability of a customer choosing one of three products.

- **Ordinal logistic regression:** This type of logistic regression is used to predict the probability of one of three or more outcomes that are ordered. For example, ordinal logistic regression can be used to predict the probability of a patient rating their pain on a scale of 1 to 10.

Logistic regression is a versatile and powerful tool for binary classification problems. It is a good choice for problems where the data is linear and the classes are well-separated.

** Obtain Conjunctive Normal Form (CNF) for the formula :

D -> (A -> (B ^ C))

To obtain the Conjunctive Normal Form (CNF) for the formula D → (A → (B ^ C)), we can follow these steps:

1. **Convert the implication (→) to disjunction (∨) with negation (¬):**
   D → (A → (B ^ C)) is equivalent to ¬D ∨ (A → (B ^ C))

2. **Convert the second implication (→) to disjunction (∨) with negation (¬):**
   ¬D ∨ (A → (B ^ C)) is equivalent to ¬D ∨ (¬A ∨ (B ^ C))

3. **Apply De Morgan's Law to distribute the negation (¬A):**
   ¬D ∨ (¬A ∨ (B ^ C)) is equivalent to (¬D ∨ ¬A) ∨ (B ^ C)

4. **Apply De Morgan's Law to distribute the negation (¬D):**
   (¬D ∨ ¬A) ∨ (B ^ C) is equivalent to ((¬D ∨ B) ∧ (¬D ∨ ¬A)) ∨ (B ^ C)

5. **Distribute the disjunction (∨) over the conjunction (∧):**
   (((¬D ∨ B) ∧ (¬D ∨ ¬A)) ∨ (B ^ C)) is equivalent to (((¬D ∨ B) ∨ (B ^ C)) ∧ ((¬D ∨ ¬A) ∨ (B ^ C)))

6. **Apply De Morgan's Law to distribute the negation (¬D):**
   (((¬D ∨ B) ∨ (B ^ C)) ∧ ((¬D ∨ ¬A) ∨ (B ^ C))) is equivalent to ((B ∨ (¬D ∧ B) ∨ C) ∧ (¬A ∨ (¬D ∧ ¬A) ∨ C))

7. **Simplify the expressions:**
   ((B ∨ (¬D ∧ B) ∨ C) ∧ (¬A ∨ (¬D ∧ ¬A) ∨ C)) is equivalent to ((B ∨ C) ∧ (¬A ∨ C)) ∧ (¬D ∨ B) ∧ (¬D ∨ ¬A)

Therefore, the Conjunctive Normal Form (CNF) for the formula D → (A → (B ^ C)) is:

```
((B ∨ C) ∧ (¬A ∨ C)) ∧ (¬D ∨ B) ∧ (¬D ∨ ¬A)
```

** Briefly discuss the following (give suitable example for each) :

(i) Rote learning

(ii) Delayed-Reinforcement learning

(i) Rote learning

Rote learning, also known as memorization, is a learning strategy that focuses on memorizing information without necessarily understanding it. It involves repetitive drills, practice, and memorization of facts, formulas, or vocabulary. While rote learning can be effective for tasks that require recalling specific information, it often fails to promote deeper understanding or the ability to apply knowledge to new situations.

Example:

Memorizing the multiplication table or a list of vocabulary words are examples of rote learning.

(ii) Delayed-Reinforcement learning

Delayed-reinforcement learning is a type of reinforcement learning where the feedback or reward is not immediate but is delayed until later in the learning process. This type of learning is particularly useful for tasks where the consequences of actions are not immediately apparent, such as in strategic games or long-term planning.

Example:

In a game of chess, the player's movements have delayed consequences as the outcome of the game might not be determined until many moves later. Learning from these delayed consequences helps the player improve their chess skills over time.

* Obtain Disjunctive Normal Form for the well form formula given below : ~ (A → (~ B ^ C))

To obtain the DNF, we will first negate the entire formula and then apply De Morgan's Laws.

Negating the formula:

```
¬(~(A → (~ B ^ C)))
```

Applying De Morgan's Laws:

```
(A ∨ (~ B ^ C)))
```

Expanding the implication:

```
(A ∨ ((¬B) ∧ C))
```

Distributing the disjunction:

```
(A ∨ ¬B) ∧ (A ∨ C)
```

Therefore, the DNF for the formula ~(A → (~ B ^ C)) is:

```
(A ∨ ¬B) ∧ (A ∨ C)
```

** Briefly discuss the relevance of resolution and unification mechanism in artificial intelligence.

Resolution and unification are two fundamental concepts in automated reasoning, a subfield of artificial intelligence (AI) that deals with the ability of machines to reason logically and solve problems. Both resolution and unification play crucial roles in enabling AI systems to perform tasks such as theorem proving, natural language processing, and logic programming.

Resolution

Resolution is a proof procedure for first-order logic, a formal system for representing and reasoning about knowledge. It is a refutation-based system, meaning that it proves a logical statement by assuming its negation and then trying to derive a contradiction. Resolution is a key component of automated theorem provers, which are programs that can automatically prove mathematical theorems.

Unification

Unification is a process for finding a substitution that makes two terms equivalent. It is used in many areas of AI, including resolution theorem proving, natural language processing, and logic programming. In resolution theorem proving, unification is used to combine clauses in a proof. In natural language processing, unification is used to match patterns in text. In logic programming, unification is used to evaluate Prolog programs.

Relevance of Resolution and Unification in AI

Resolution and unification are relevant to AI because they provide powerful tools for automating reasoning. Resolution is a very general and powerful proof procedure that can be used to prove a wide variety of theorems. Unification is a versatile tool that can be used for a variety of tasks, such as pattern matching and variable assignment.

Here are some specific examples of how resolution and unification are used in AI:

- Theorem proving: Resolution is used to prove mathematical theorems in automated theorem provers.
- Natural language processing: Unification is used to match patterns in text, such as noun phrases and verb phrases. This is used to perform tasks such as parsing sentences and extracting information from text.
- Logic programming: Unification is used to evaluate Prolog programs. Prolog is a programming language that is based on logic, and unification is used to evaluate Prolog programs by matching Prolog goals with Prolog clauses.

In conclusion, resolution and unification are two fundamental concepts in automated reasoning that are relevant to a variety of AI tasks. Resolution is a powerful proof procedure that can be used to prove a wide variety of theorems. Unification is a

versatile tool that can be used for a variety of tasks, such as pattern matching and variable assignment.

**Pincer Search

Pincer Search

The Pincer-Search algorithm is different from the Apriori algorithm in that it finds the frequent itemset in a bidirectional approach.

In each iteration, Pincer-Search algorithm checks the support count of frequent itemsets and prunes the infrequent itemsets.

Pincer-Search algorithm is more efficient than Apriori algorithm when the frequent itemset is long. This is because Pincer-Search algorithm does not need to maintain the support score of subsets of frequent itemsets.

Feature	Pincer-Search algorithm	Apriori algorithm
Approach	Bidirectional	Bottom-up
Pruning	Yes	No
Efficiency	Typically more efficient for long frequent itemsets	Typically less efficient for long frequent itemsets

14.4.3 Generating Association Rules using Frequent Itemset

Once the frequent itemsets F(k) are generated from set of transactions T, next step is to generate strong association rules from them. Recall that *strong* association rules satisfy both minimum support as well as minimum confidence. Theoretically, confidence is defined as the ratio of frequency of transactions containing items x and *y* to the frequency of transactions that contained item *x*.

Based on the definition. Follow the given rules to generate strong association rules:

1. For each itemset f ϵ F(k), generate subsets of f.
2. For every non-empty subset $x \epsilon f$, generate the rule $x => f - x$

$$\frac{support(f)}{support(x)} \geq min\_conf$$

Consider the set of transactions and the generated frequent itemsets described in section 1.3.2. One of the frequent itemset *f* belonging to set F(3) is:

$$f = \{Bread, Butter, Cookies\}.$$

Following the steps given above, the non-empty subsets $x \subseteq F(3)$,

$$x = \{\{Bread\}, \{Butter\}, \{Cookies\}, \{Bread, Butter\}, \{Bread, Cookies\}, \{Butter, Cookies\}\}$$

where all itemsets are sorted in lexicographic order. The generated association rules with their

15.6 Clustering Algorithms

a) K-Means

Among clustering algorithms, is an algorithm that tries to minimize the distance of the points in a cluster with their *centroid* – the k-means clustering technique.

K-means is a centroid-based algorithm. It can also be called asa distance-based technique where distances between points are calculated to allocate a point to a cluster. Each cluster in K-Means is paired with a centroid.

The K-Means algorithm's main goal is to reduce the sum of distances between points and their corresponding cluster centroid.

Real World Example:

Let's have a look at an example. Assume you went to a bookstore to purchase some books. There are several types of books that can be found there. One thing you'll notice is that the books are sorted into groups based on their category. All of the literature books will be kept in one location, while science books will be organized by type. The K-Means algorithm's main goal is to reduce the sum of distances between points and their corresponding cluster centroid.

Now we will understand this with the help of these figures.

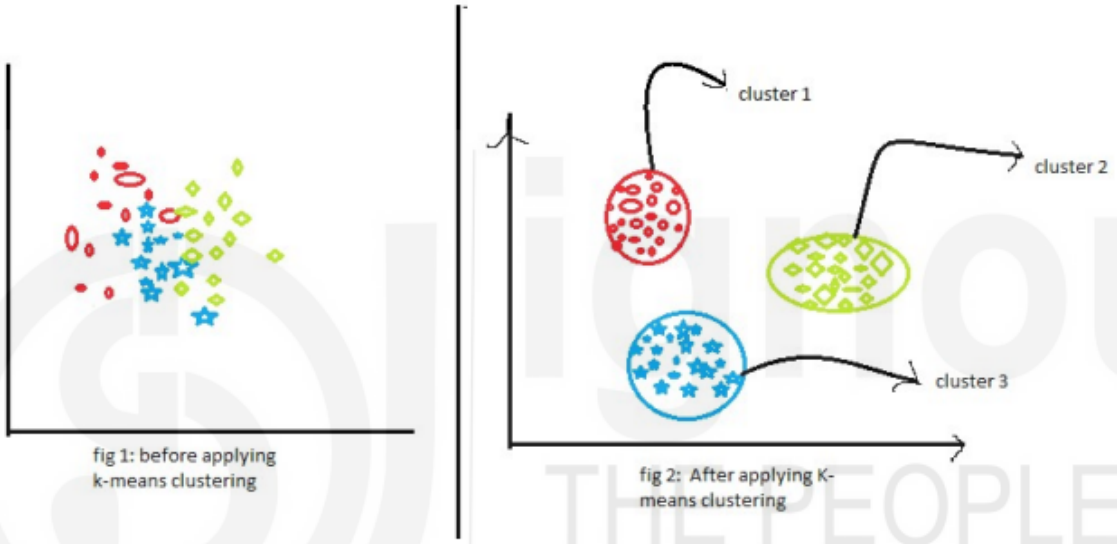


Fig 5. Before and after K-means clustering

In the figure 5 data is presented into two stages. The first figure shows the data in raw stage which is not clustered by k-means. Here all types of data are clubbed together thus becomes impossible to differentiate them into their original category.

The second figure shows three clusters of three different colors red, green, and blue. These clusters are formed after applying k-means clustering. The second figure shows data into three different categories which are called clusters.

Working of K-means clustering algorithm .

k-means clustering technique is an immense clustering algorithm to group similar types of data in groups which are so called clusters. It is the simplest and commonly used technique in machine learning extensively used for data analysis. It can easily locate the similarity points

1. Selection of k values.
2. Initialization of the centroids.
3. Selection of the cluster and finding the average.

Let us understand the above steps with the help of the Fig6:

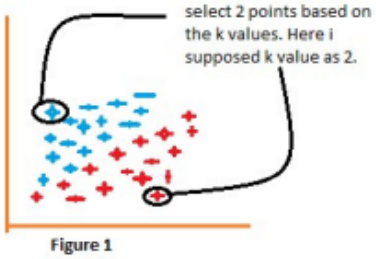
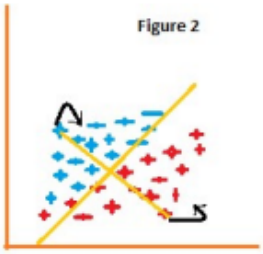
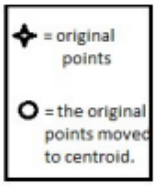
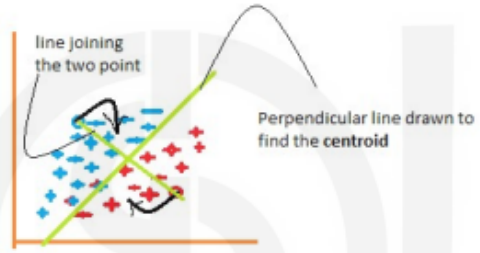


Figure 1





F2: Find the average of all the blue points and red points and move the selected points to **centroid.**



F3: Some of the red points changed to blue points, that means they belong to the group blue now. Again the repeat the same process.

F4: The same process has been applied here. This process will be continued until we get the **two complete different cluster.**

2. Confusion Matrix:

- The confusion matrix tells us how well the model works and gives us a matrix or table as Output.

- Sometimes, this kind of structure is called the error matrix.
- The matrix is a summary of the results of the predictions. It shows how many predictions were right and how many were wrong.

The matrix looks like as below table:

The matrix looks like as below table:

Actual Class \ Predicted Class	Positive	Negative	
Positive	True Positive (TP)	False Negative (FN) **Type II Error**	**Sensitivity** $\frac{TP}{(TP+FN)}$
Negative	False Positive (FP) **Type I Error**	True Negative (TN)	**Specificity** $\frac{TN}{(TN+FP)}$
	Precision $\frac{TP}{(TP+FP)}$	**Negative Predictive Value** $\frac{TN}{(TN+FN)}$	**Accuracy** $\frac{TP+TN}{(TP+TN+FP+FN)}$

Naive Bayes Algorithm

The Naive Bayes algorithm is a simple and effective probabilistic classifier based on Bayes' theorem. It assumes that features are independent of each other, which is rarely true in practice, but despite this assumption, it often works well in practice. The Naive Bayes algorithm is commonly used for text classification, spam filtering, and medical diagnosis.

Key Steps of Naive Bayes Algorithm

1. Training: Estimate the prior probabilities of the classes and the conditional probabilities of the features given the classes.
2. Classification: Calculate the probability of each class for a given data point using Bayes' theorem and assign the class with the highest probability to the data point.

Types of Naive Bayes Algorithms

1. Gaussian Naive Bayes: Assumes that the features have a Gaussian distribution.
2. Multinomial Naive Bayes: Assumes that the features are discrete and follow a multinomial distribution.

Applications of Naive Bayes Algorithm

1. Text classification: Classifying text documents into different categories, such as spam, news, or sports.
2. Spam filtering: Identifying spam emails.
3. Medical diagnosis: Diagnosing medical conditions based on symptoms and test results.

Pros of Naive Bayes Algorithm

1. Simple and easy to understand.
2. Efficient to train and classify data.
3. Performs well in practice, even though it makes the assumption of feature independence.

Cons of Naive Bayes Algorithm

1. Assumption of feature independence may not be realistic.
2. Sensitive to outliers and missing values.
3. May not perform well when the distribution of the features is not Gaussian or multinomial.

K-Nearest Neighbors (K-NN)

The K-Nearest Neighbors (K-NN) algorithm is a non-parametric classification algorithm that classifies new data points based on the majority class of its k nearest neighbors in the training set.

Key Steps of K-NN Algorithm

1. Select the value of k: Choose the number of nearest neighbors to consider for classification.
2. Calculate distances: Compute the distance between the new data point and all data points in the training set.
3. Identify k nearest neighbors: Find the k data points in the training set that are closest to the new data point.
4. Determine majority class: Assign the class that is most frequent among the k nearest neighbors to the new data point.

Applications of K-NN Algorithm

- Pattern recognition: Identifying patterns in data, such as image recognition or spam filtering.
- Recommendation systems: Recommending products or content to users based on their past preferences and similarities to other users.
- Medical diagnosis: Assisting in medical diagnosis by comparing patient data to similar cases.

Pros of K-NN Algorithm

- Simple and easy to understand.
- Effective for both classification and regression tasks.
- Does not require any assumptions about the underlying data distribution.

Cons of K-NN Algorithm

- Performance can be sensitive to outliers and noise in the data.
- Computational complexity increases with the size of the training data.
- Sensitive to the choice of the k value.

Algorithmic Clustering

Clustering is a fundamental unsupervised learning technique that aims to group data points into clusters based on their inherent similarities. Clustering algorithms are primarily classified into two categories: agglomerative clustering and divisive clustering.

Agglomerative Clustering

Agglomerative clustering, also known as bottom-up clustering, follows a hierarchical approach. It starts with each data point as a separate cluster and iteratively merges the most similar clusters until a predefined number of clusters or a stopping criterion is reached. The process of merging clusters is often visualized using a dendrogram, a tree-like diagram that represents the merging hierarchy.

Steps of Agglomerative Clustering:

1. Initialize each data point as a separate cluster.
2. Calculate the pairwise distances between all clusters.
3. Identify the two closest clusters.
4. Merge the two closest clusters into a single cluster.
5. Repeat steps 2-4 until the desired number of clusters is reached or a stopping criterion is met.

Applications of Agglomerative Clustering:

- Customer segmentation: Identifying groups of customers with similar characteristics for targeted marketing campaigns.

- Gene expression analysis: Grouping genes with similar expression patterns to identify potential biomarkers or disease-related genes.
- Image segmentation: Grouping pixels in an image based on their color or intensity for object recognition.

Pros of Agglomerative Clustering:

- Simple and easy to understand.
- Efficient for dealing with large datasets.
- Produces a hierarchical representation of the data.

Cons of Agglomerative Clustering:

- Sensitive to the choice of distance metric.
- Merging decisions cannot be undone once made.
- May not capture non-hierarchical relationships between data points.

Divisive Clustering

Divisive clustering, also known as top-down clustering, takes the opposite approach of agglomerative clustering. It starts with all data points in a single cluster and iteratively splits the cluster into smaller subclusters until each data point is in its own cluster. The splitting process is often guided by a similarity measure or a stopping criterion.

Steps of Divisive Clustering:

1. Initialize all data points as a single cluster.
2. Calculate a similarity measure for the current cluster.
3. If the similarity measure is below a threshold or a stopping criterion is met, stop splitting.
4. Identify the most dissimilar data points within the current cluster.
5. Split the current cluster into two subclusters, each containing one of the most dissimilar data points.
6. Repeat steps 2-5 for each of the newly formed subclusters.

Applications of Divisive Clustering:

- Concept learning: Identifying concepts or categories in a dataset.

- Anomaly detection: Identifying data points that deviate significantly from the rest of the data.
- Medical diagnosis: Grouping patients with similar symptoms for further diagnosis.

Pros of Divisive Clustering:

- Captures non-hierarchical relationships between data points.
- Avoids making irreversible merging decisions.
- May be more efficient for certain types of data.

Cons of Divisive Clustering:

- Complex and computationally expensive.
- Does not produce a hierarchical representation of the data.
- May lead to over-clustering if not carefully controlled.

In summary, both agglomerative clustering and divisive clustering are effective techniques for grouping data points based on their similarities. The choice of algorithm depends on the specific characteristics of the data, the desired number of clusters, and the computational resources available.